

Mercari 가격 예측 문제에서는 문맥 이해 기반 임베딩보다 브랜드명, 모델명, 옵션과 같은 핵심 키워드 및 카테고리 정보가 가격 결정에 더 큰 영향을 미친다. 본 장에서는 이러한 데이터 특성을 반영하여 토큰 기반 벡터화와 앙상블 모델을 중심으로 최적의 예측 성능을 도출하였다.

## 5 장

# MERICARI

## Price

## Suggestion

---

## 제 5 장 목차

|                                                    |    |
|----------------------------------------------------|----|
| 5. MERICARI Price Suggestion(가격 예측) .....          | 2  |
| 5.1 프로젝트 개요 및 이론적 배경 .....                         | 2  |
| 5.1.1 개요 .....                                     | 2  |
| 5.1.2 이론적 배경 .....                                 | 2  |
| 5.1.3 프로젝트 정의 및 목표 .....                           | 3  |
| 5.1.4 머신 러닝 이론 .....                               | 4  |
| 5.2 데이터 분석 및 전처리 단계 (EDA & Preprocessing) .....    | 6  |
| 5.2.1 데이터 이해 및 EDA .....                           | 6  |
| 5.2.2 피처 엔지니어링 (Feature Engineering) .....         | 11 |
| 5.2.3 텍스트 벡터화 전략의 진화 .....                         | 17 |
| 5.3 모델링 및 최적화 전략 .....                             | 21 |
| 5.3.1 모델 선정 및 탐색 .....                             | 21 |
| 5.3.2 PyCaret, AutoGluon, Scikit-learn 비교 개요 ..... | 26 |
| 5.3.3 하이퍼파라미터 튜닝 .....                             | 42 |
| 5.3.4 시스템 안정성 및 효율성 확보 .....                       | 42 |
| 5.4 모델 평가 및 결론 .....                               | 43 |
| 5.4.1 모델 평가 및 지표 해석 .....                          | 43 |
| 5.4.2 최종 모델 선정 및 앙상블 전략 .....                      | 44 |
| 5.4.3 결론 및 시사점 .....                               | 49 |

## 5. MERCARI Price Suggestion(가격 예측)

### 5.1 프로젝트 개요 및 이론적 배경

#### 5.1.1 개요

Mercari Price Suggestion Challenge 는 중고 거래 플랫폼 **Mercari** 에 등록된 상품 정보를 기반으로 각 상품의 판매 가격을 예측하는 머신 러닝 문제를 다룬다.

상품명, 상품 설명과 같은 비정형 텍스트 데이터와 카테고리, 브랜드, 배송 여부 등의 정형 데이터가 함께 제공되며, 실제 전자상거래 환경에서 발생하는 가격 결정 문제를 반영하고 있다.

본 프로젝트에서 사용한 Mercari 학습 데이터는 총 **1,482,535 개의 상품**으로 구성되어 있으며, 상품명, 카테고리, 브랜드, 상품 설명과 같은 텍스트 정보와 상품 상태(item\_condition\_id), 배송 여부(shipping) 등의 정형 변수를 포함한다. 목표 변수는 **상품의 판매 가격(price)**이다.

본 프로젝트는 해당 데이터를 활용하여 상품의 다양한 속성을 종합적으로 고려한 **가격 예측 모델**을 구축하는 것을 목표로 한다.

#### 5.1.2 이론적 배경

##### 5.1.2.1 가격 예측 문제의 특성

중고 상품 가격은 다음과 같은 특성을 가진다.

- 가격 분포가 **오른쪽으로 치우친 비대칭 분포**
- 동일 카테고리 내에서도 **설명 방식 및 브랜드에 따른 가격 편차**
- **저가 상품이 다수를 차지**하고, 고가 상품은 소수 존재

이로 인해 단순 평균 기반 회귀 모델은 고가 상품 예측에서 상대적으로 큰 오차를 발생시키기 쉽다.

### 5.1.2.2 텍스트 기반 피처의 중요성

Mercari 데이터에서는 상품의 상태나 특징이 정형 컬럼보다 텍스트 설명에 더 많이 포함되어 있다.

- **TF-IDF, 임베딩(BERT, Word2vec 등)**과 같은 자연어 처리 기반 텍스트 표현 기법이 가격 예측 성능에 중요한 역할을 하며, 텍스트 길이, 단어 수, 브랜드 언급 여부와 같은 **텍스트 통계 피처** 또한 보조적인 설명력을 제공한다.

### 5.1.2.3 RMSLE 평가 지표의 이론적 근거

Mercari 대회에서는 가격의 로그 스케일 오차를 측정하는 RMSLE(Root Mean Squared Logarithmic Error)를 공식 평가 지표로 사용한다.

RMSLE 는

- **상대적으로 오차에 민감**
- 고가 상품에서의 절대 오차 과대평가 완화
- 저가 상품 예측의 정확도를 중요하게 반영

하는 특성이 있어, 본 문제의 데이터 분포 특성에 적합한 평가 지표이다.

## 5.1.3 프로젝트 정의 및 목표

### 5.1.3.1 프로젝트 정의

본 프로젝트는 Mercari 플랫폼에 등록된 상품 정보를 입력으로 받아 각 상품의 **판매 가격을 예측하는 회귀 기반 머신 러닝** 문제로 정의한다. 텍스트 데이터와 범주형·수치형 데이터가 혼합된 환경에서 데이터 누수를 방지하고, 실제 서비스 환경에서도 적용할 수 있는 가격 예측 모델을 구축하는 것을 문제의 핵심으로 설정한다.

### 5.1.3.2 문제 유형 및 목표

본 프로젝트는 **지도학습 기반 회귀(Regression) 문제**로, 상품의 다양한 입력 피처를 활용하여 연속형 값인 **상품 판매 가격을 예측**하는 것을 목표로 한다. 상품명과 상품 설명으로부터 추출한 **텍스트 피처**와 카테고리, 브랜드, 배송 여부, 상품 상태 등의 구조적 피처를 효과적으로 결합하여, 실제 가격과 예측 가격 간의 차이를 최소화하는 모델을 구축한다.

특히, 가격 분포의 특성을 고려하여 Kaggle Mercari 대회의 공식 기준과 동일한 RMSLE 평가 지표를 최소화하는 것을 본 프로젝트의 **최종 목표**로 설정한다.

## 5.1.4 머신 러닝 이론

### 5.1.4.1 회귀 모델

본 프로젝트는 상품 가격이라는 **연속형 값을 예측**하는 문제 특성을 고려하여, **회귀 트리(Tree-based Regression)** 기반 모델을 중심으로 구성하였다. 특히 **LightGBM, XGBoost, CatBoost** 와 같은 그래디언트 부스팅 계열 모델은 다음과 같은 장점을 가진다.

- 비선형 관계를 효과적으로 학습 가능
- 텍스트 벡터화 결과와 범주형·수치형 피처가 혼합된 고차원 데이터에 적합
- 이상치(outlier)에 대한 비교적 강인한 성능
- 대규모 데이터셋에서도 안정적인 학습 속도

이러한 특성으로 인해, Mercari 데이터와 같이 **텍스트 정보와 구조적 피처가 결합된 가격 예측 문제**에 회귀 트리 기반 모델이 적합하다고 판단하였다.

#### 5.1.4.2 평가 지표

Mercari Price suggestion Challenge에서는 가격 예측 성능을 평가하기 위해 RMSLE를 공식 지표로 사용한다. RMSLE는 실제 가격과 예측 가격을 로그 스케일로 변환한 후 평균 제곱 오차의 제곱근을 계산하는 지표로, 다음과 같이 정의된다.

$$\text{RMSLE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\log(1 + \hat{y}_i) - \log(1 + y_i))^2}$$

[그림 5.1.1 RMSLE 공식]

이 지표는

- 상대적으로 오차에 민감
- 고가 상품에서의 절대 오차 과대평가 완화
- 저가 상품 예측의 정확도를 중요하게 반영

특성을 지니므로, 가격 분포가 비대칭적인 Mercari 데이터에 적합한 평가 기준이다.

#### 5.1.4.3 최적화 기법

회귀 트리 기반 모델은 하이퍼파라미터 설정에 따라 성능 차이가 크게 발생한다. 이에 따라 본 프로젝트에서는 모델 성능을 효율적으로 개선하기 위해 **Optuna 기반**

**베이지안 최적화(Bayesian Optimization) 기법**을 적용하였다. 베이지안 최적화는 이전 실험 결과를 기반으로 다음 탐색 지점을 확률적으로 선택함으로써, 무작위 탐색이나 그리드 탐색 대비 적은 반복 횟수로도 효율적인 하이퍼파라미터 탐색이 가능하다.

Optuna의 TPE(Tree-structured Parzen Estimator) 알고리즘을 활용하여 탐색 비용을 최소화하면서도 RMSLE 기준 성능이 우수한 하이퍼파라미터 조합을 도출하고자 하였다.

## 5.2 데이터 분석 및 전처리 단계 (EDA & Preprocessing)

본 프로젝트는 Mercari 상품의 정형(상품상태, 배송여부 등) + 비정형(상품명/설명 텍스트) 정보를 함께 활용해 price(연속값)를 예측하는 회귀 문제이다. 데이터는 train.tsv / test.tsv 형태로 제공되며, 주요 컬럼은 **name**, **item\_condition\_id**, **category\_name**, **brand\_name**, **shipping**, **item\_description**, 그리고 타겟 **price** 로 구성된다. 데이터 속성표는 아래와 같다.

| 속성명               | 설명                       |
|-------------------|--------------------------|
| train_id          | 데이터 ID                   |
| name              | 제품명                      |
| item_condition_id | 판매자가 제공하는 제품 상태          |
| category_name     | 카테고리 명                   |
| brand_name        | 브랜드 이름                   |
| price             | 제품 가격 (예측을 위한 타겟 변수)     |
| shipping          | 배송비 무료 여부 (1: 무료, 0: 유료) |
| item_description  | 제품에 대한 상세 설명             |

또한 **train.tsv** 는 약 330MB 규모로 로컬 환경에서 메모리 부담이 큰 편이어서, 전처리 캐싱 및 샘플링(undersampling) 전략을 함께 사용했다.

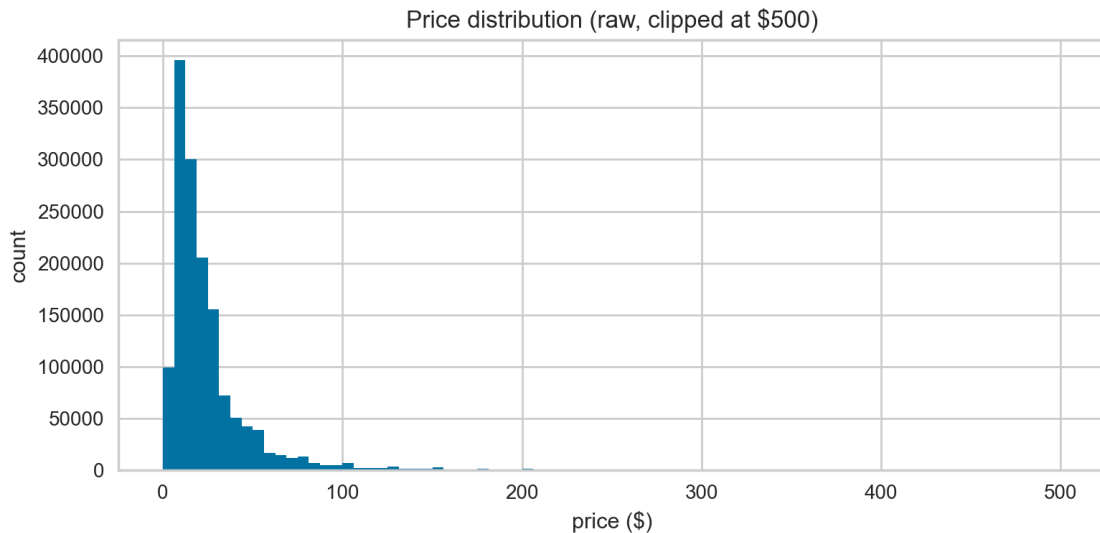
### 5.2.1 데이터 이해 및 EDA

#### 5.2.1.1 타겟 변수 분석: price 변수의 심한 왜곡(Skewness) 확인

Mercari 데이터의 **price** 는 저가 상품이 매우 많고, 고가 상품은 소수로 길게 꼬리가 늘어지는 형태(Heavy-tail)를 보인다. 이러한 분포는 원 스케일에서 회귀 모델이 고가

구간의 오차에 과민 해지거나, 저가 구간을 과도하게 “0 근처로” 끌어당기는 문제를 유발할 수 있다.

### 분석 포인트



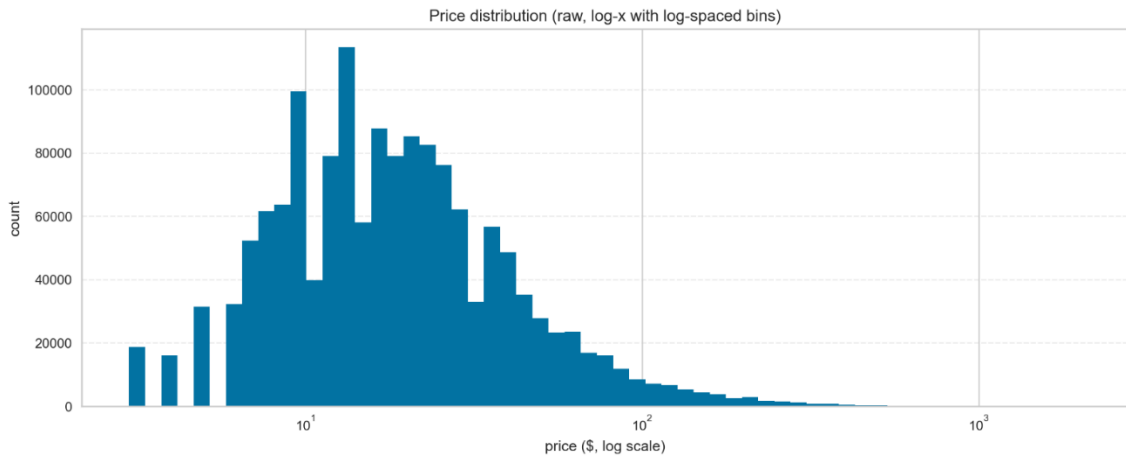
[그림 5.2.1 원본 price 히스토그램]

- 원본 price 는 **저가 구간(대략 0~50 달러)에 관측치가 극도로 집중되어 있고, 고가 상품은 소수만 존재하는 롱테일(long-tail) 구조를 보인다.** 상한을 500 달러로 clip 한 이유는, 극단적인 고가(outlier)가 x 축 범위를 과도하게 늘려 **대다수 구간의 분포 형태가 보이지 않는 문제**를 방지하기 위함이다.

→ 따라서 “원본 타겟은 심한 양의 왜도(skewness)를 가지며, 그대로 회귀를 학습할 경우 고가 소수 샘플의 영향이 커지고 오차 분산이 커질 수 있음”을 근거로 제시하기 좋다.

- 왜도(skewness) 확인 및 고가 outlier 존재 여부 확인





[그림 5.2.2 원본 price 로그축 히스토그램]

### 5.2.1.2 로그 변환: $\log(x+1)$ 변환 적용

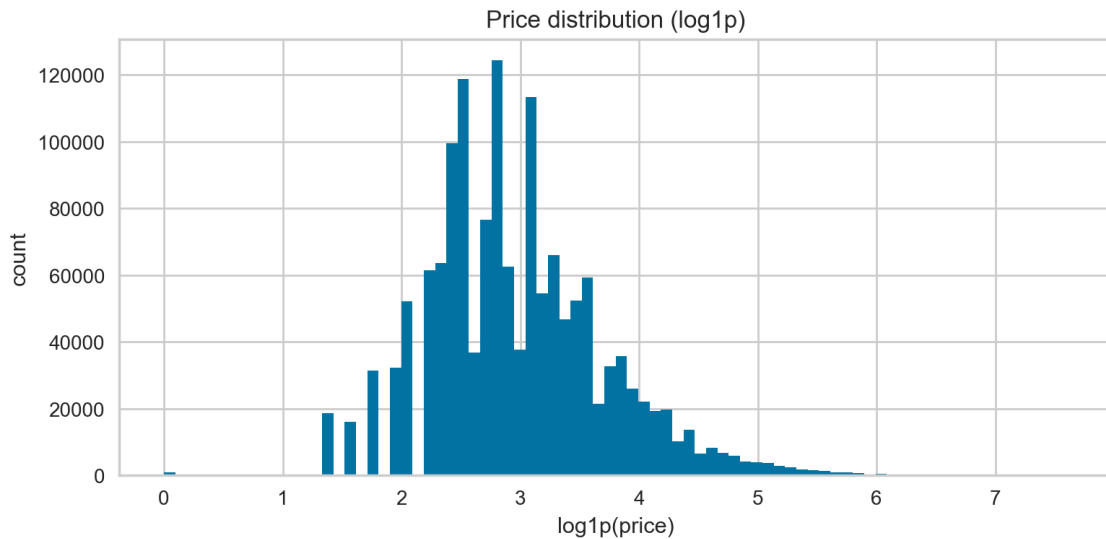
본 프로젝트는 대회 공식 지표가 **RMSLE** 이기 때문에, 타겟을  **$\log_{1p}(\text{price})$** 로 변환하여 학습하는 것이 목적함수와 더 자연스럽게 정렬된다. 또한 로그 변환은 **고가 구간의 폭발적인 스케일**을 눌러주어 모델이 저가~중가 구간 패턴을 안정적으로 학습하도록 돕는다.

#### 구현 방식(코드 흐름)

- PyCaret 기반 파이프라인 기반으로 price > 0 필터 후 price =  $\log_{1p}(\text{price})$ 로 변환
- sklearn 파이프라인에서도 y\_log =  $\log_{1p}(\text{price})$ 로 동일하게 타겟을 로그 스케일로 통일

#### 효과

- 분포 정규성 개선 → 회귀 학습 안정화
- RMSLE 평가와 목표 정렬(로그 공간 오차 최소화)



[그림 5.2.3 price 로그변환 히스토그램]

- **log1p(price)** 적용 후 분포는 원본 대비 **왜도가 완화**되고, 회귀 모델이 학습하기 더 쉬운 형태에 가까워진다. 또한 Mercari 평가지표가 RMSLE 계열이기 때문에, 타겟을 log 로 변환하면 **모델이 '상대적 오차'를 더 안정적으로 학습하는 효과가 있다.**
- 막대가 특정 구간에서 붐우리가 생기는 것은, 실거래에서 **\*\*선호 가격대(예: \$10, \$15, \$20 등)\*\***가 반복적으로 등장해 로그 공간에서도 군집이 생기기 때문이다.

### 5.2.1.3 결측치 처리: 범주형/텍스트 피쳐 결측치 처리 방법론

Mercari 데이터는 **brand\_name, category\_name, item\_description, name** 등에서 결측치가 발생한다. 결측치를 그대로 두면 벡터라이저/인코더 단계에서 오류 또는 희소 행렬 품질 저하가 발생하므로 명시적인 **placeholder** 로 통일했다.

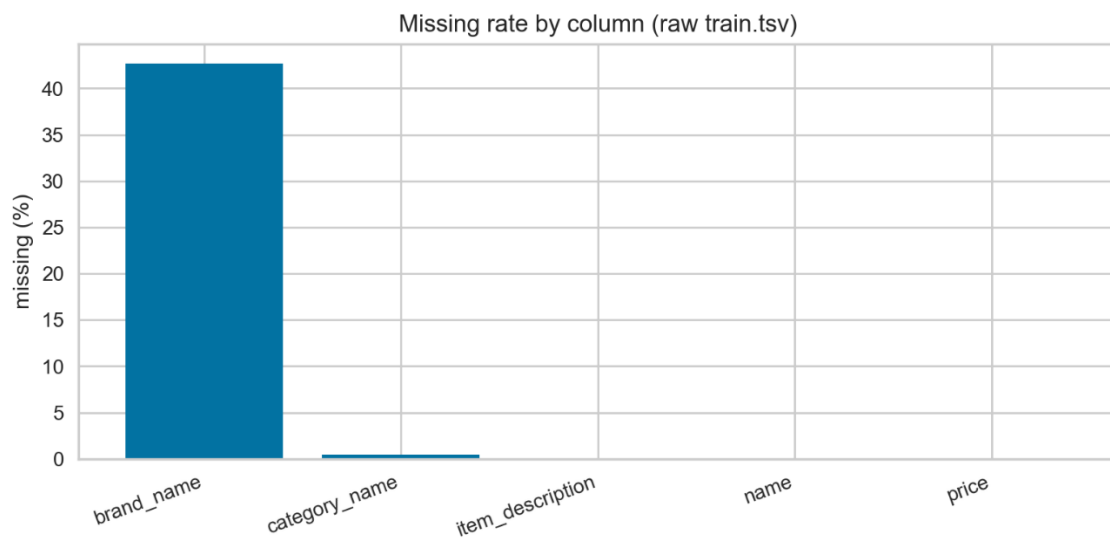
#### 결측치 처리 전략

- **brand\_name**: "missing" 또는 "Unknown"

- **category\_name**: "NoCat/NoCat/NoCat" 혹은 "Unknown"
- **item\_description**: "No description"
- **name**: "No name"

### 기대 효과

- 전처리 파이프라인 안정화(인코더/벡터화 단계 오류 방지)
- 결측 자체가 신호가 될 수 있어 모델이 "정보 부재" 패턴을 학습 가능



[그림 5.2.4 결측치 처리]

- **brand\_name** 결측이 압도적으로 높게 나타나는 것은, **판매자가 브랜드를 입력하지 않는 경우가 매우 많기 때문**이고(현실 데이터 특성), 텍스트 기반 가격예측 문제에서 흔하다.
- **category\_name, item\_description** 은 일부 결측이 존재하며, **name, price** 는 상대적으로 결측이 거의 없다(또는 없다).
- 이 그래프는 "전처리로 결측을 채우기 전에" 원본 문제를 보여주므로, **결측 처리 전략의 타당성**(Unknown/No description 등으로 대체)을 설명하는 근거로 적합하다.

## 5.2.2 피처 엔지니어링 (Feature Engineering)

### 5.2.2.1 범주형 피처 처리: category\_name 3 단계 분리 및 희소(Rare) 처리

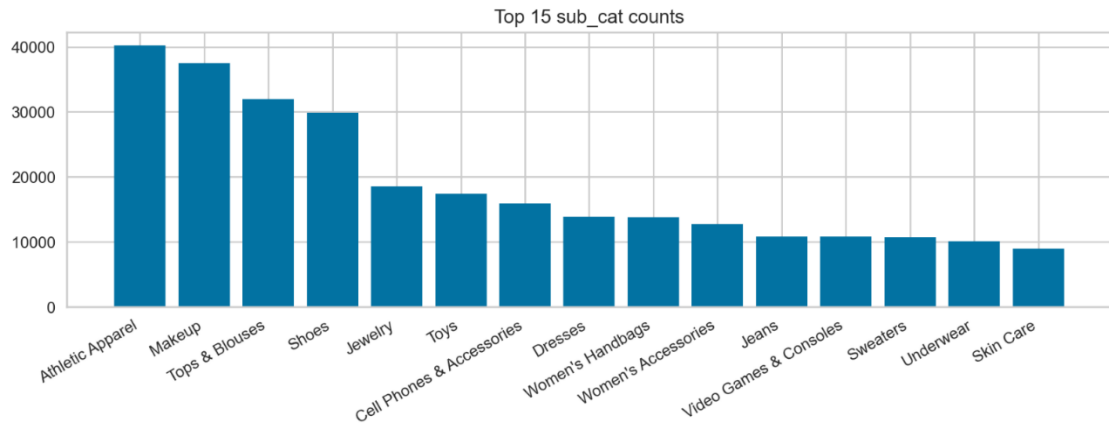
category\_name 은 "대/중/소" 구조의 계층 정보를 내포하므로 '/' 기준 분리하여 main\_cat, sub\_cat, sub\_sub\_cat 로 확장했다. 또한 브랜드/카테고리는 유니크 값이 매우 많아 희소도가 높으므로, 빈도 기반 희소값 추약 및 희소 플래그를 적용했다.

#### 구현 포인트

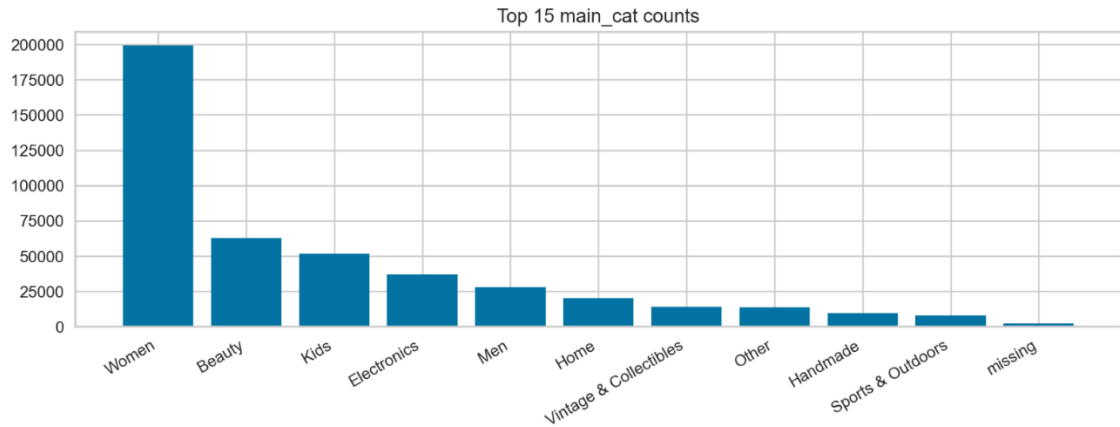
- 카테고리 3 분할: category\_name → main\_cat/sub\_cat/sub\_sub\_cat
- 희소값 추약: 상위 top\_k 외 값은 "Other\_\*"로 통합
  - ✓ 예: brand\_name(top 5000), main/sub/sub\_sub(top 1000) 유지, 나머지는 Other 처리
- 희소 플래그: rare\_brand, rare\_main\_cat, rare\_sub\_cat, rare\_sub\_sub\_cat 등

#### 효과

- 인코딩 공간 폭발 완화(메모리/시간 절감)
- "드문 브랜드/카테고리"가 가격 예측에 주는 영향(불확실성)을 모델이 별도 학습 가능

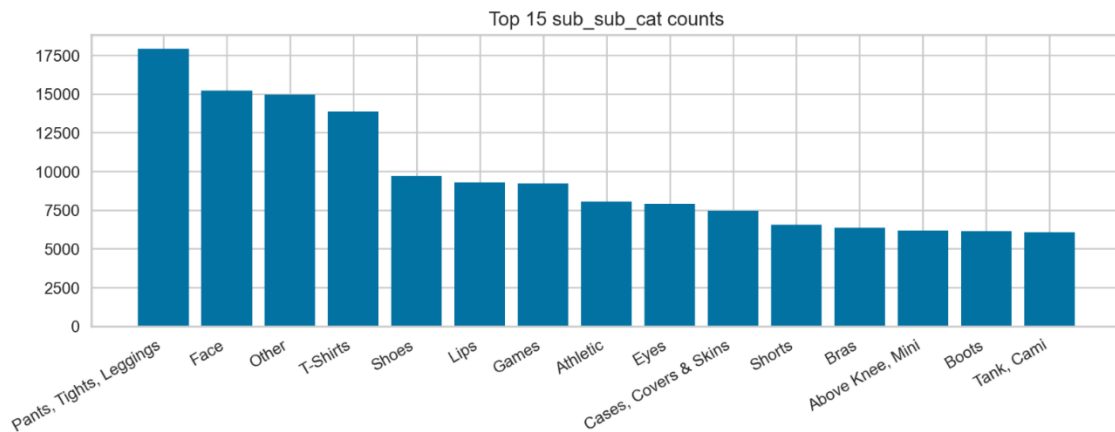


[그림 5.2.5 범주형 피쳐]



[그림 5.2.6 범주형 피쳐]

- **main\_cat** 에서 Women 이 압도적으로 크고, Beauty/Kids/Electronics... 순으로 이어지는 것은, 플랫폼의 상품 구성 자체가 특정 대분류에 편중되어 있음을 의미한다.



[그림 5.2.7 범주형 피쳐]

- sub\_cat, sub\_sub\_cat 로 갈수록 롱테일이 더 강해지고(종류가 폭증), 상위 몇 개를 제외하면 관측치가 급격히 작아진다.

→ 이 특성 때문에 “희소(rare) 카테고리 처리”가 중요해지고, 단순 원-핫/라벨인코딩만으로는 정보손실 또는 과적합 위험이 커진다.

### 5.2.2.2 텍스트 통계 피쳐: 상품명/설명 길이, 단어 수 등

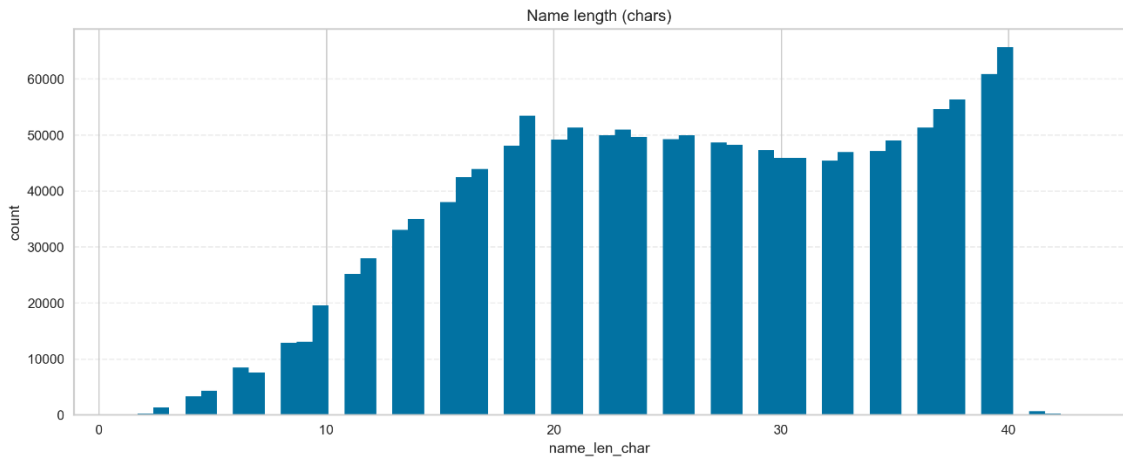
텍스트 벡터(TF-IDF/임베딩)가 내용 의미를 담는다면, 텍스트 통계 피쳐는 **품질·정보량·노이즈 정도**를 직접 반영한다. 버전 92 에서는 아래와 같은 통계 피쳐가 추가되어 있다.

#### 추가 피쳐 예시

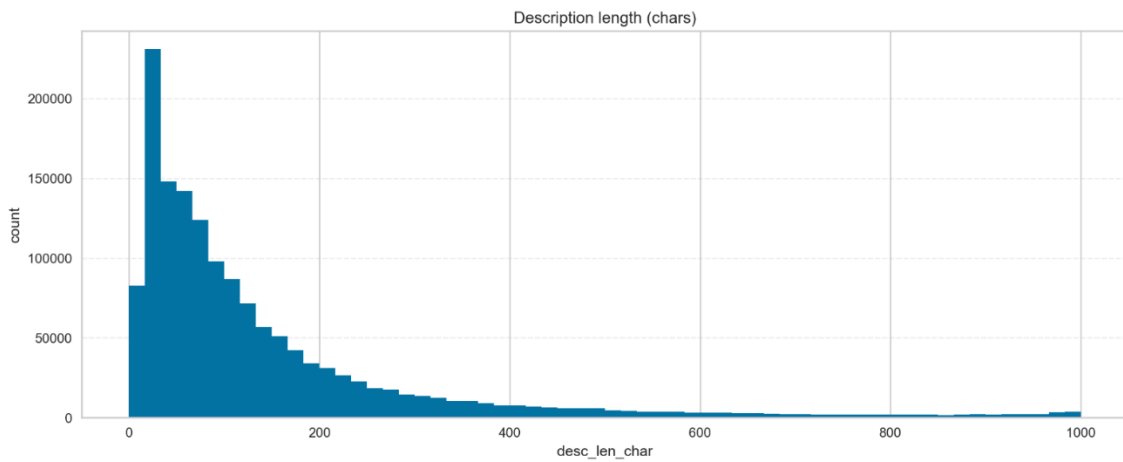
- **name\_len\_char, desc\_len\_char** : 문자 길이
- **name\_len\_word, desc\_len\_word** : 단어 수
- **brand\_in\_name** : 브랜드명이 title 에 포함되는지 여부(신뢰도/정품/검색성 신호)

#### 효과

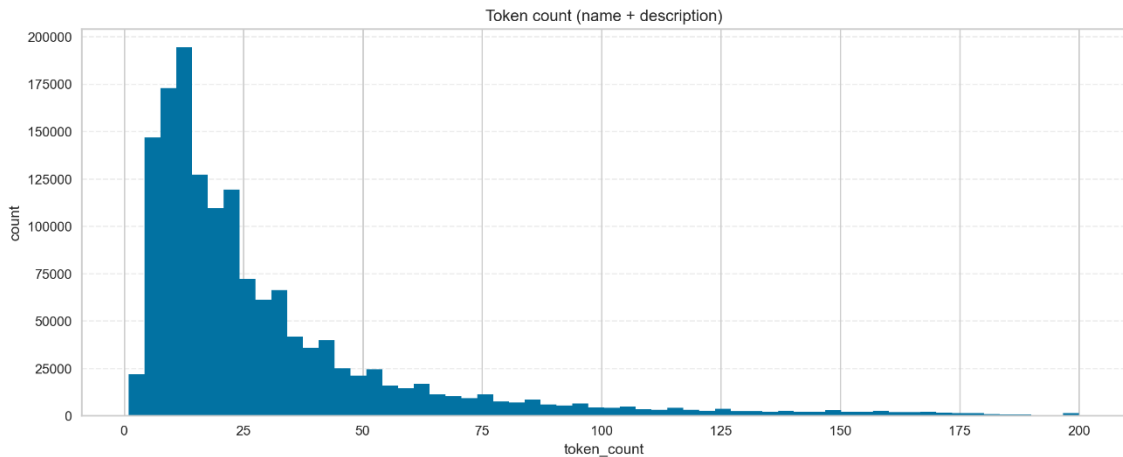
- “설명 없음/짧은 글” vs “상세 설명”의 가격 패턴 차이를 학습
- 단순하지만 강한 보조 신호로 작동(특히 저가/중가 구간)



[그림 5.2.8 텍스트 통계 피쳐]



[그림 5.2.9 텍스트 통계 피쳐]



[그림 5.2.10 텍스트 통계 피처]

- 상품명(name)과 설명(item\_description)은 길이 분포가 **짧은 문장에 집중되고 일부 샘플이 매우 긴 롱테일을 보인다**. 이에 따라 텍스트 벡터화(TF-IDF/FastText/BERT 등)와 별개로, **문자길이/단어 수/토큰수**와 같은 "텍스트 통계 피처"를 추가하여 상품 정보량을 수치로 반영하였다.
- 텍스트 통계 피처는 특히 "짧은 설명 vs 상세 설명"의 차이를 모델이 쉽게 학습하도록 돕고, 극단적으로 긴 설명은 clip 을 적용해 분포 왜곡을 완화하였다.

### 5.2.2.3 SAFE 그룹 통계: Train 데이터 기반 브랜드/카테고리 가격 통계 (Data Leakage 방지)

가격 예측에서 브랜드/카테고리별 평균 가격은 강력한 신호지만, 잘못 만들면 데이터 누수가 발생한다. 따라서 **Train 데이터만으로 그룹 통계를 만들고**, 이를 Train/Test 에 머지하여 SAFE 하게 사용했다.

#### 추출 통계

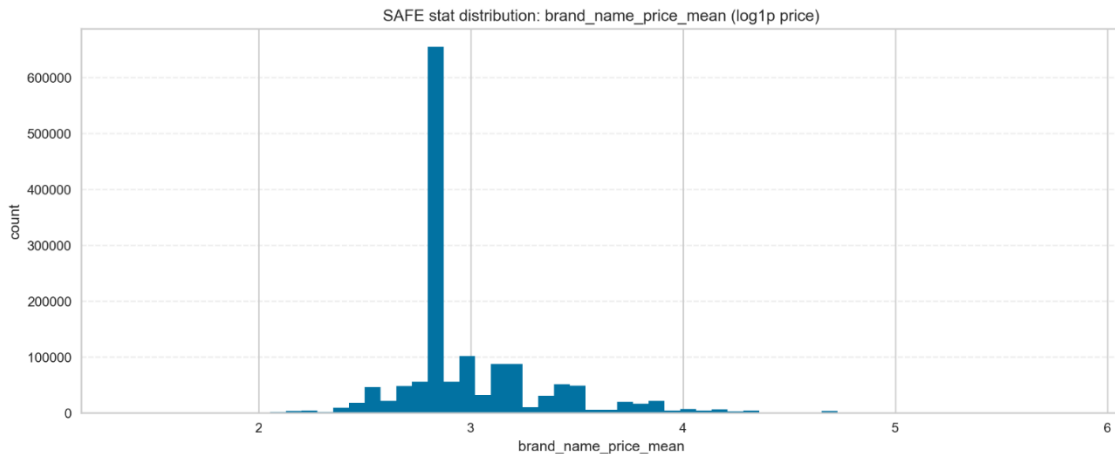
- 브랜드/카테고리별 **price\_mean, price\_median, price\_std**
- (Train 전용) **brand\_price\_z = (price - mean\_brand\_price) / std\_brand\_price**



- Test 는 **price** 가 없으므로 **brand\_price\_z** 는 0 등으로 처리.

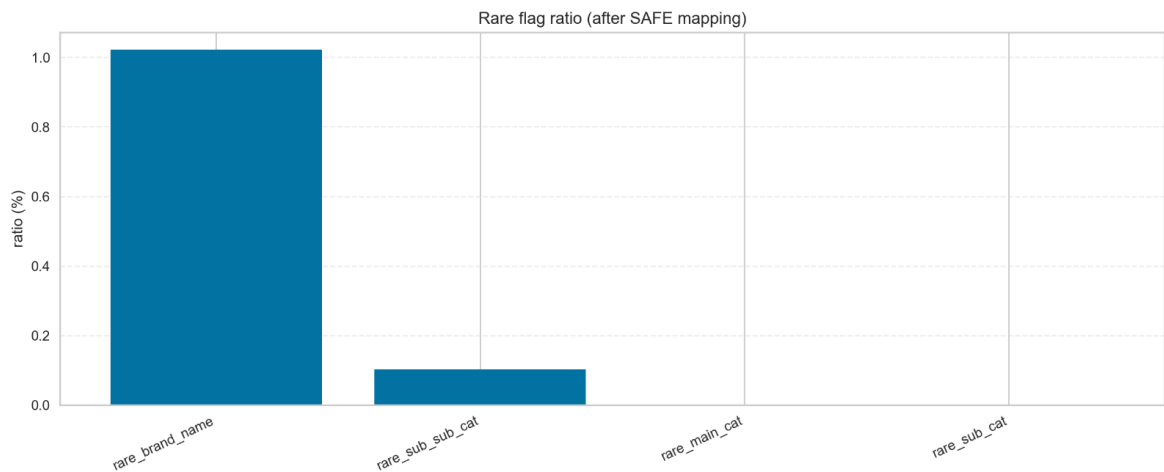
### 효과

- 강력한 prior(사전 평균) 제공 → 예측 안정화
- 누수 방지 원칙 준수(통계 생성에 Test/Valid 타겟을 사용하지 않음)



[그림 5.2.11 SAFE 그룹]

- 브랜드/카테고리별로 가격 수준이 다르다는 점을 반영하기 위해, Train 데이터만 사용하여 \*\*그룹별 price 통계(평균/중앙값/표준편차)\*\*를 계산한 뒤 각 샘플에 매핑하였다. 이때 **Test 정보를 사용하지 않으므로 Train→Test 방향의 Data Leakage**를 방지할 수 있다(단, 엄밀한 의미의 leakage-free 검증을 위해서는 CV 기반(out-of-fold) 통계가 더 안전함).



[그림 5.2.12 SAFE 그룹]

- 희소 그룹(등장 빈도 < threshold)은 통계값이 불안정하므로 **rare flag** 를 추가하고, 해당 그룹 통계는 기본값(전역 평균 등)으로 보정하였다.

## 5.2.3 텍스트 벡터화 전략의 진화

### 5.2.3.1 V4 (메모리 효율성): HashingVectorizer → 성능 저하

- 목적: TF-IDF 희소행렬의 메모리 부담을 줄이기 위해 Hashing 기반으로 고정 차원 벡터화를 검토/시도
- 결과: 정보 손실/해시 충돌로 **성능( $R^2$ /RMSLE) 저하**가 발생하여 유지하지 않음

**주의(내가 모르는 부분):** 업로드된 v92 코드에서는 HashingVectorizer 구현이 "직접적으로" 확인되지 않았고, 문서/회의 맥락에서 "시도한 전략"으로 언급된 성격이 강해. 보고서에는 **"\*\*"실험/검토 단계에서 성능 저하로 배제"**처럼 보수적으로 쓰는 게 안전하다고 판단되었다.

### 5.2.3.2 V5 (성능 복구): TF-IDF 복귀 및 max\_features 제한

- TF-IDF 로 복귀하되, **max\_features** 제한 + **min\_df/max\_df/sublinear\_tf** 등으로 차원을 제어
- 추가로 **TruncatedSVD** 로 차원 축소하여 **메모리/학습시간을 통제하면서도 성능을 회복**
- TF-IDF + (SVD) 기반 흐름이 안정적인 기본 베이스라인으로 유지됨

### 5.2.3.3 V6-V9 고급 벡터화 실험 및 베이스라인 선정

대용량 데이터 환경에서는 텍스트 벡터화 단계가 전체 파이프라인에서 가장 큰 계산 비용을 차지한다. 이에 따라 본 연구에서는 다양한 임베딩 기법을 비교하되, 캐싱 및 샘플링 전략을 적용하여 반복 실험에 따른 계산 비용을 최소화하는 방향으로 분석을 확장하였다. 본 절에서는 V6-V9 단계에서 수행한 고급 벡터화 실험의 범위와 결과를 정리하고, 최종 베이스라인 벡터화 방식을 선정한 근거를 제시한다.

#### 1) 실험 범위 및 최적화 전략

V6-V9 단계에서는 다음과 같은 벡터화 기법을 대상으로 비교 실험을 수행하였다.

- **TF-IDF (SVD 포함, CPU/GPU 변형)**
- Word2Vec / GloVe / FastText 계열 임베딩
- BERT 및 SentenceTransformer 기반 문장 임베딩

대용량 데이터 환경에서의 실험 효율성을 확보하기 위해 다음과 같은 최적화 전략을 적용하였다.

- **캐싱:** 벡터화 결과를 ../results/cache 에 저장하여 재실행 비용 최소화
- **샘플링:** Train 데이터 일부(약 30%)로 사전 비교 후, 유망한 조합만 전체 학습으로 확장

- **차원 축소:** SVD 차원 수를 조절하여 성능과 자원 사용 간 균형 최적화

## 2) 임베딩 방식별 성능 비교 결과

TF-IDF 기반 벡터화의 주요 성능 지표는 다음과 같다.

$R^2 = 0.9923$ ,  $RMSE = 3.3996$ ,  $MAE = 0.1727$ ,  $RMSLE(\text{Kaggle 기준}) = 0.01243$ .

| Embedding | RMSE | RMSLE   |
|-----------|------|---------|
| TF-IDF    | 3.39 | 0.01242 |
| FastText  | 3.42 | 0.01276 |
| Word2Vec  | 3.59 | 0.01288 |
| BERT      | 3.78 | 0.01389 |

비교 결과, **TF-IDF > FastText > Word2Vec > BERT** 순으로 성능이 나타났다.

## 3) 결과 해석 및 베이스라인 선정 근거

가격 예측 문제에서 핵심적으로 작용하는 정보는 브랜드명, 모델명, 시리즈 번호, 옵션 정보와 같은 토큰 단위 특징이다. Mercari 데이터는 중고 상품 중심으로 구성되어 있어, 특정 희귀 토큰의 존재 여부가 가격 결정에 직접적인 영향을 미친다.

이러한 특성으로 인해 원본 토큰의 출현 빈도를 직접 반영하는 TF-IDF 방식이 문맥 기반 임베딩(FastText, BERT)보다 높은 설명력을 보인 것으로 판단된다. 특히 BERT 임베딩은 고차원 밀집 벡터 구조로 인해, 희소 고차원 입력에 강점을 가지는 LightGBM 계열 모델과의 결합에서 상대적으로 불리하게 작용하였다.

**4) 소결**

V6-V9 고급 벡터화 실험을 통해, 본 문제에서는 의미 기반 임베딩보다 토큰 기반 벡터화가 가격 예측에 더 효과적임을 확인하였다. 이에 따라 이후 단계에서는 TF-IDF(SVD)를 최종 베이스라인 벡터화 방식으로 채택하고, 이를 기반으로 앙상블 모델 설계를 진행하였다.

## 5.3 모델링 및 최적화 전략

### 5.3.1 모델 선정 및 탐색

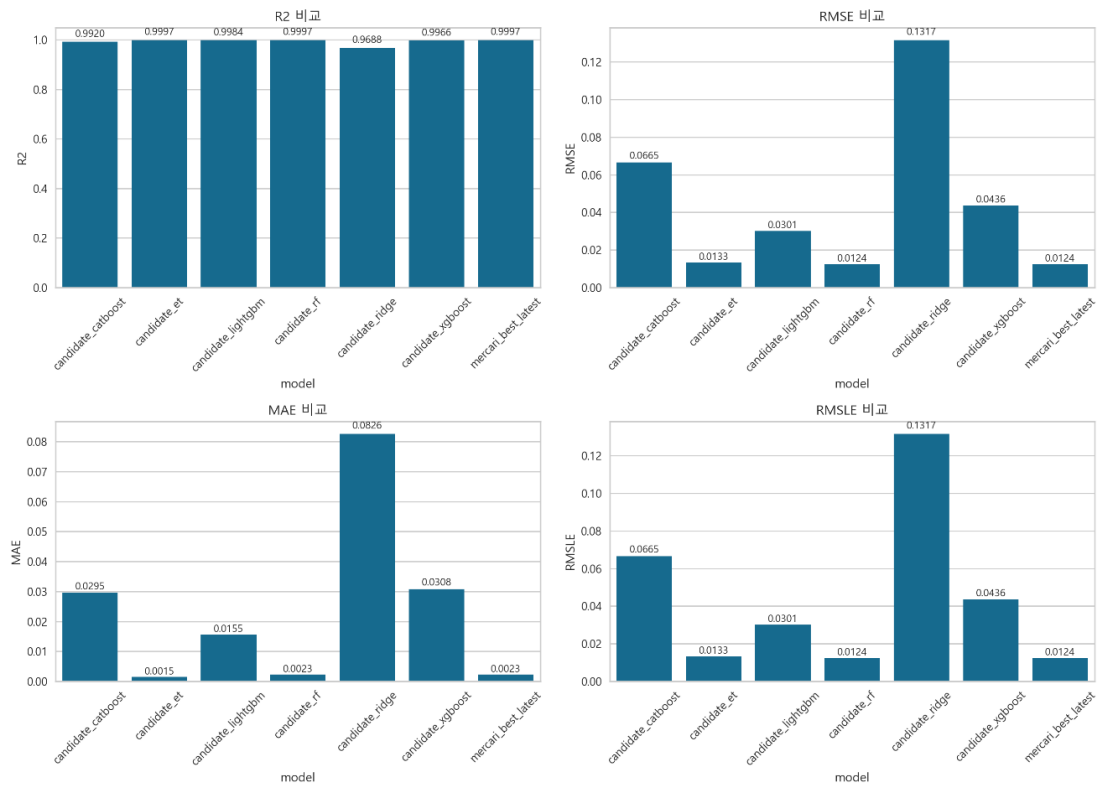
#### 5.3.1.1 Kaggle Winners 기반 후보 모델 선별

모델 탐색 단계에서는 탐색 공간을 무작위로 확장하기보다, Kaggle 경진대회에서 반복적으로 우수한 성능이 검증된 알고리즘을 중심으로 후보 모델을 선별하였다. 이는 모델 탐색 시간을 효율적으로 관리함과 동시에, 가격 예측 문제에서 높은 성능을 기대할 수 있는 모델에 분석 자원을 집중하기 위한 전략이다.

선별 기준은 다음과 같다. ① 대용량 데이터 처리 가능성, ② 비선형 관계 학습 능력, ③ 실무 및 경진대회에서의 성능 검증 이력, ④ 앙상블 구성 시 상호 보완 가능성이다.

| 모델명              | 모델 유형               | 선정 사유                                              |
|------------------|---------------------|----------------------------------------------------|
| LightGBM (LGBM)  | Gradient Boosting   | 속도와 메모리 효율이 우수하며, 대용량 데이터에서 안정적인 성능을 보이는 부스팅 모델    |
| XGBoost (XGB)    | Gradient Boosting   | 정교한 규제와 안정적인 학습 구조를 통해 다양한 데이터셋에서 강력한 성능을 보이는 모델   |
| CatBoost (CB)    | Gradient Boosting   | 범주형 변수 인코딩을 내부적으로 처리하여 전처리 부담이 적고, 과적합에 강한 특성을 보임  |
| Ridge Regression | Linear Model        | 학습 속도가 빠르고 대용량 데이터에 적합한 선형 모델로, 신뢰할 수 있는 베이스라인 제공  |
| Extra Trees (ET) | Tree-based Ensemble | 무작위 분할을 활용하여 분산을 줄이고 과적합에 강하며, 앙상블 구성 시 안정성 향상에 기여 |

이와 같이 선정된 5 개 모델은 서로 다른 학습 구조와 가정을 기반으로 하고 있어, 개별 성능 비교뿐 아니라 스택킹 및 블렌딩과 같은 앙상블 기법 적용 시 상호 보완적인 효과를 기대할 수 있다. 이후 절에서는 해당 후보 모델을 대상으로 개별 성능 평가를 수행하고, 최적의 조합을 도출하는 과정을 설명한다.



[그림 5.2.13 모델별 오차 지표]

### 5.3.1.2 모델 선정 지표

본 절에서는 Kaggle Winners 기반으로 선별된 후보 모델들을 대상으로, 가격 예측 성능을 정량적으로 비교하기 위해  $R^2$ , RMSE, MAE, RMSLE 지표를 활용하였다. 각 지표는 모델의 설명력과 오차 특성을 서로 다른 관점에서 평가하며, 최종 모델 선정을 위한 핵심 판단 기준으로 사용되었다.

#### 성능 지표 설명

| 지표    | 의미                                    | 해석 기준   |
|-------|---------------------------------------|---------|
| $R^2$ | 모델이 실제값의 분산을 얼마나 잘 설명하는지를 나타내는 설명력 지표 | 높을수록 우수 |

|       |                                            |         |
|-------|--------------------------------------------|---------|
| RMSE  | 예측값과 실제값 간 평균 제곱 오차의 제곱근으로 큰 오차에 민감함       | 낮을수록 우수 |
| MAE   | 예측값과 실제값의 절대 오차 평균으로 직관적인 오차 크기 지표         | 낮을수록 우수 |
| RMSLE | 로그 변환된 예측값과 실제값 간 오차로, 가격 예측에서 상대적 오차에 민감함 | 낮을수록 우수 |

### 1) $R^2$ (결정 계수) 비교

$R^2$ 는 모델이 종속 변수의 분산을 얼마나 잘 설명하는지를 나타내는 지표로, 값이 1에 가까울수록 설명력이 우수함을 의미한다. 실험 결과 대부분의 후보 모델이 0.99 이상의 매우 높은  $R^2$  값을 기록하였다. 특히 candidate\_et와 mercari\_best\_latest 모델은 0.9997로 가장 높은 설명력을 보였다. 반면 candidate\_ridge 모델은 0.9888로 상대적으로 낮은 값을 기록하여, 비선형 관계를 충분히 반영하는 데 한계가 있음을 시사한다.

### 2) RMSE (평균 제곱근 오차) 비교

RMSE는 예측값과 실제값 간의 평균적인 오차 크기를 나타내는 지표로, 값이 낮을수록 모델의 예측 정확도가 높음을 의미한다. candidate\_et와 candidate\_rf 모델은 각각 0.0133과 0.0124로 가장 낮은 RMSE를 기록하였다. mercari\_best\_latest 모델 역시 0.0124로 매우 우수한 성능을 보였다. 반면 candidate\_ridge 모델은 0.1317로 다른 모델들에 비해 현저히 큰 오차를 보여, 가격 예측 문제에서의 한계를 명확히 드러냈다.

### 3) MAE (평균 절대 오차) 비교

MAE는 예측 오차의 절대적인 크기를 측정하는 지표로, 이상치의 영향에 상대적으로 덜 민감하다는 특징을 가진다. candidate\_et 모델은 0.0015로 가장 낮은 MAE를 기록하여, 전반적으로 가장 안정적인 예측 성능을 보였다. candidate\_rf 모델이



0.0023 으로 그 뒤를 이었다. candidate\_ridge 모델은 0.0826 으로 가장 큰 오차를 기록하여, RMSE 결과와 동일한 경향을 나타냈다.

#### 4) RMSLE (평균 제곱근 로그 오차) 비교

RMSLE 는 실제값이 0 또는 작은 양수일 때의 예측 오차에 민감하게 반응하는 지표로, 가격 예측과 같이 분포의 비대칭성이 큰 문제에서 중요하게 활용된다. 특히 과소 예측에 대한 패널티가 크다는 특징을 가진다. 실험 결과 candidate\_et 와 candidate\_rf 모델이 가장 낮은 RMSLE 를 기록하여, 가격 예측 성능이 우수함을 입증하였다. candidate\_ridge 모델은 0.1317 로 가장 높은 RMSLE 를 기록하였다.

#### 5) 종합 결론 및 최종 후보 모델 선정

종합적으로 살펴보면, Extra Trees(candidate\_et) 및 Random Forest(candidate\_rf) 계열 모델이  $R^2$  기준의 높은 설명력을 유지하면서도 RMSE, MAE, RMSLE 전 지표에서 가장 낮은 오차를 기록하였다. 이는 해당 모델들이 가격 예측 문제에서 단일 모델 기준으로 가장 우수한 성능을 보였음을 의미한다. 따라서 이후 단계에서는 이들 모델을 핵심 베이스 모델로 활용하고, 부스팅 계열 모델과의 결합을 통한 스택킹 앙상블 구성으로 확장하는 전략을 채택하였다.

##### 5.3.1.3 Stacking 설계 이유 (LGBM + XGBoost + Extra Trees)

본 연구에서는 단일 모델의 성능 한계를 극복하고 예측 안정성을 향상시키기 위해 스택킹(Stacking) 앙상블 기법을 적용하였다. 스택킹은 서로 다른 학습 특성을 가진 여러 베이스 모델의 예측 결과를 결합함으로써, 개별 모델의 편향과 분산을 동시에 완화할 수 있는 효과적인 앙상블 전략이다.

본 스택킹 구조에서는 LightGBM(LGBM), XGBoost(XGB), Extra Trees(ET)를 베이스 모델로 구성하였다. 해당 모델들은 모두 트리 기반 알고리즘이지만, 학습 방식과 분할 전략, 규제 구조에서 차이를 가지며 서로 다른 오류 패턴을 보인다.

| 베이스 모델           | 학습 특성              | 스태킹에서의 역할            |
|------------------|--------------------|----------------------|
| LightGBM (LGBM)  | 리프 중심 분할, 빠른 학습 속도 | 대용량 데이터에서 전반적인 패턴 학습 |
| XGBoost (XGB)    | 정교한 규제와 안정적 부스팅    | 복잡한 비선형 관계 보완        |
| Extra Trees (ET) | 무작위 분할 기반, 낮은 분산   | 예측 안정성 및 과적합 완화      |

이와 같은 구성은 부스팅 계열 모델(LGBM, XGB)의 강력한 학습 능력과 Extra Trees의 분산 감소 효과를 결합함으로써, 가격 예측 문제에서 높은 정확도와 안정성을 동시에 확보하기 위한 설계이다.

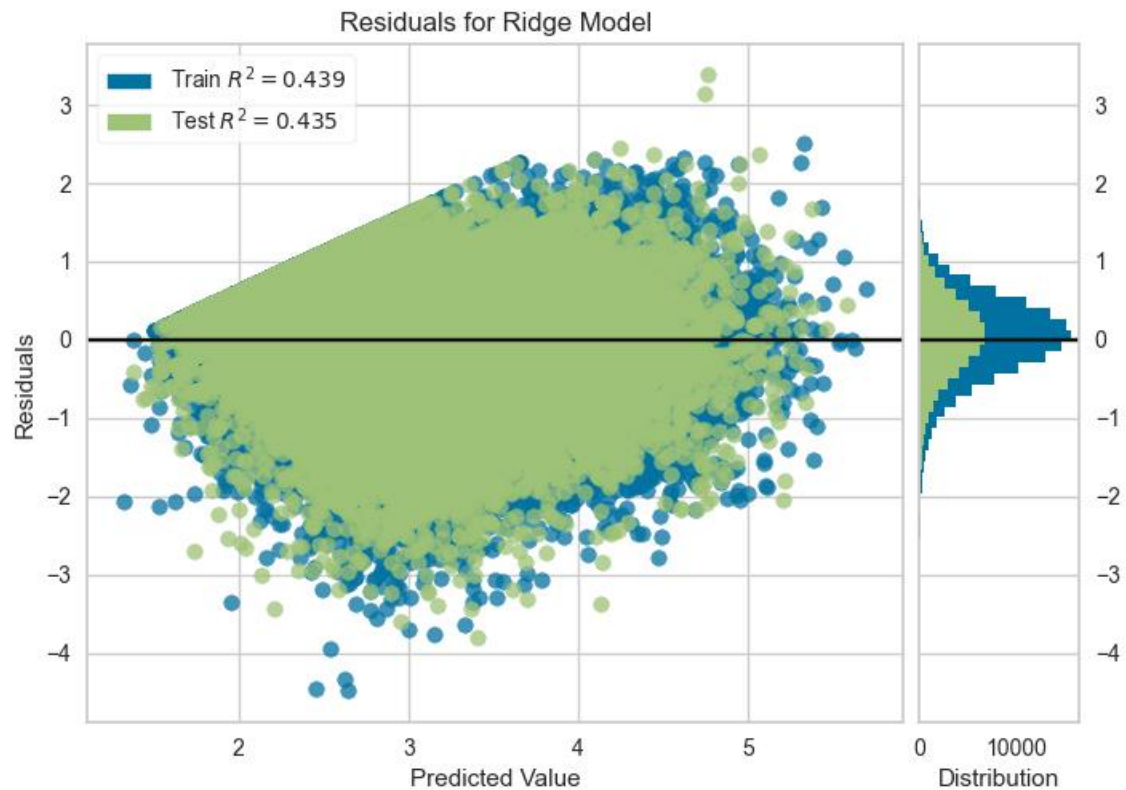
#### 5.3.1.4 Ridge 를 Meta-Learner 로 선택한 이유

스태킹 앙상블의 최종 단계에서는 베이스 모델들의 예측 결과를 입력으로 받아 최종 예측값을 산출하는 메타 학습기(meta-learner)가 필요하다. 본 연구에서는 Ridge Regression 을 메타 러너로 선택하였다.

Ridge Regression 은 L2 정규화를 포함한 선형 모델로, 베이스 모델 예측값 간의 선형 결합을 안정적으로 학습할 수 있다는 장점을 가진다. 특히 베이스 모델 간 예측 상관관계가 높은 스태킹 구조에서, 과도한 가중치 집중을 방지하는 데 효과적이다.

또한 Ridge 는 학습 속도가 매우 빠르고 하이퍼파라미터가 단순하여, 대용량 데이터 환경에서도 추가적인 계산 부담 없이 적용할 수 있다. 이는 복잡한 비선형 메타 모델을 사용하는 경우 발생할 수 있는 과적합 위험과 재현성 저하 문제를 최소화하는 데 기여한다.

결과적으로 Ridge 기반 메타 러너는 ① 베이스 모델 예측의 선형 조합을 통한 안정적인 일반화, ② 과적합 억제, ③ 해석 가능성 확보라는 측면에서 본 스태킹 구조에 가장 적합한 선택으로 판단되었다.



[그림 5.3.2 Ridge 메타 러너의 잔차 분포 분석]

Ridge 회귀 기반 메타 러너의 잔차 분포를 분석한 결과, 학습 데이터와 테스트 데이터 간 분포 차이가 크지 않으며 안정적인 일반화 성능을 보였다.

### 5.3.2 PyCaret, AutoGluon, Scikit-learn 비교 개요

이 프로젝트는 모델링 자동화, 성능 최적화, 유연성 확보라는 세 가지 목표를 위해 다양한 프레임워크를 실험하고 비교했다.

| 프레임워크   | 역할 및 특징                          | 주요 이점                             |
|---------|----------------------------------|-----------------------------------|
| pycaret | 자동화된 ML 실험 관리, 모델 비교, 하이퍼파라미터 튜닝 | 신속한 모델 비교 및 단일 모델 최적화 (V8 전략의 핵심) |

|              |                                   |                                        |
|--------------|-----------------------------------|----------------------------------------|
| autogluon    | 완전 자동화된 ML 파이프라인 및 앙상블(Stacking)  | 높은 잠재 성능 확보, FE/모델링 완전 자동화 (V7 전략의 핵심) |
| scikit-learn | 기본 모델 및 수동 파이프라인 구축, 커스텀 튜닝 엔진 통합 | 유연한 사용자 정의, 파라미터 튜닝 엔진 통합 용이성          |

### 5.3.2.1 PyCaret 기반의 단일 모델 최적화 전략

PyCaret 은 머신 러닝 워크플로우를 자동화하고 효율화하는 Low-Code 라이브러리이다. 본 전략의 핵심 엔진으로서, 복잡한 앙상블 대신 단일 고성능 모델(LGBM, CatBoost 등)의 빠르고 투명한 실험, 비교, 최적화에 초점을 맞추는 데 사용되었다. 특히 GPU 가속 기능을 활용하여 실험 속도를 극대화했다.

| 역할    | 주요 기능                                     | 효과                                                |
|-------|-------------------------------------------|---------------------------------------------------|
| 핵심 엔진 | setup_pycaret, compare_models, tune_model | 일관된 환경에서 모델 간 성능을 투명하게 비교하고, 최적 하이퍼파라미터를 신속하게 도출  |
| 기술 활용 | GPU 가속 (LGBM, CatBoost 등)                 | 대규모 데이터셋에서 모델 학습 및 튜닝 시간을 획기적으로 단축                |
| 통합    | 텍스트 벡터화 및 피처 엔지니어링 결과 결합                  | 텍스트(TF-IDF SVD)와 구조적(범주형/수치형) 피처를 통합하여 모델 입력으로 사용 |

### 코드 해석 및 PyCaret 워크플로우

데이터 전처리→텍스트 벡터화 →피처 결합→PyCaret 설정의 순서로 진행된다.

데이터 전처리 및 피처 엔지니어링 (\_stratified\_sample, normalize\_text, build\_text\_stats, build\_interactions)

| 함수                 | 목적               | 주요 로직                                                                                                                              |
|--------------------|------------------|------------------------------------------------------------------------------------------------------------------------------------|
| _stratified_sample | 가격 기반 계층적 언더 샘플링 | 훈련 데이터의 목표 변수(price)를 10 개(bins=10) 구간으로 나누고, 각 구간에서 일정 비율(frac=0.3)의 샘플만 추출하여 훈련 속도를 높이고 가격 분포의 균형을 유지 (⚠ stratified undersample) |

|                    |              |                                                                                              |
|--------------------|--------------|----------------------------------------------------------------------------------------------|
| normalize_text     | 텍스트 정규화      | name, item_description 컬럼의 텍스트를 소문자화하고, 숫자를 num 으로, 특수문자를 공백으로 치환하여 텍스트 정제 ( normalize_text) |
| build_text_stats   | 텍스트 통계 피처 추출 | 정규화된 텍스트에서 토큰 개수, 고유 토큰 개수, 평균 단어 길이 등 수치형 통계량을 계산하여 구조적 피처로 활용 (text_stats)                 |
| build_interactions | 상호작용 피처 생성   | brand_name 과 main_cat 을 결합하여 새로운 범주형 피처 brand_main 을 생성 (PyCaret setup 시 범주형으로 처리됨)          |

텍스트 벡터화 및 차원 축소 (vectorize\_text\_tfidf, vectorize\_text\_tfidf\_gpu, vectorize\_text)

이 단계는 PyCaret 에 투입할 핵심 특징 벡터를 생성한다.

**vectorize\_text\_tfidf (CPU 기반):** name 과 item\_description 을 각각 TfidfVectorizer 로 희소 행렬로 변환한다.

각 희소 행렬을 TruncatedSVD 를 사용하여 n\_components=150 차원으로 축소한다.

축소된 두 결과를 수평으로 결합하여 최종 텍스트 특징 벡터를 생성한다.

차원 축소 단계(TruncatedSVD)에서 cuML 라이브러리를 사용하여 GPU 가속을 시도하고, 실패 시 sklearn 의 CPU 기반 SVD 로 대체하는 폴백(Fallback) 로직을 포함한다.

**vectorize\_text (통합 인터페이스):** 외부 캐시 파일(vectorized\_{method}\_\*.pkl) 존재 여부를 먼저 확인한다.

self.use\_gpu 설정에 따라 GPU 전용 메서드를 우선 호출하며, 실패 시 또는 GPU 기능이 없을 시 CPU 메서드를 호출한다.

**피처 결합 (\_add\_categorical\_numeric\_features):**

PyCaret setup 전에, 텍스트 벡터화로 생성된 self.train\_vectorized 와 기존 구조적 피처를 하나로 합치는 단계이다.

범주형 피처 처리: main\_cat, brand\_name 등 범주형 피처는 Label Encoding 을 통해 수치형으로 변환된다. 이 때, 훈련/테스트 셋을 합쳐 인코더를 fit 하여 누락된 범주(Category)를 방지한다.

**수치형 피처 결합:** name\_tok\_count, brand\_name\_price\_mean 등 이전에 추출된 텍스트 통계 및 집계 기반 피처들이 벡터화된 데이터 프레임에 추가된다.

PyCaret 설정 (setup\_pycaret)

최종 준비된 데이터셋을 PyCaret 환경에 로드한다.

```
self.setup_result = setup(
    data=df,
    target="price",
    session_id=session_id,
    fold_strategy="kfold",
    fold=fold,
    categorical_features=categorical or None,
    normalize=True, # 모든 피처를 정규화
    use_gpu=self.use_gpu, # GPU 사용 설정
    n_jobs=n_jobs,
    verbose=True,
)
```

**Target:** price (회귀 문제로 설정)

**Fold:** K-Fold 교차 검증(fold=3) 사용.

**Categorical Features:** 결합된 범주형 피처 리스트를 전달하여 PyCaret 이 내부적으로 One-Hot Encoding 등 추가 전처리를 수행하도록 한다.

**Normalize:** 모든 피처를 정규화하여 모델 학습 안정성을 높인다.

**use\_gpu:** GPU 가속 기능을 활성화하여 compare\_models() 실행 시 LGBM, CatBoost 등 GPU 를 지원하는 모델의 학습 속도를 높인다.

### 5.3.2.2 AutoGluon 기반의 자동화된 앙상블 전략

#### 1) 적절한 성능, 빠른 학습

| 역할           | 주요 목표                                                          | 사용 설정                                                      |
|--------------|----------------------------------------------------------------|------------------------------------------------------------|
| 블랙박스형<br>앙상블 | 복잡한 전처리 및 스택킹(Stacking)을 자동화하여 최고 성능의 예측 모델을 신속하게 도출           | presets="medium_quality_faster_train",<br>time_limit=600 초 |
| 평가지표<br>최적화  | 대회의 핵심 평가지표인 RMSLE (Root Mean Squared Log Error)에 집중하여 모델을 최적화 | eval_metric="rmse" (AutoGluon 의 로그 변환을 통해 RMSLE 효과 달성)     |

#### 가. AutoGluon 모델 학습 개요

- 모델: TabularPredictor
- 문제 유형: regression (회귀)
- 평가 지표: rmse (Root Mean Squared Error)
- 프리셋: medium\_quality\_faster\_train
- 학습 시간 제한: 600 초 (10 분)
- 저장 경로: autogluon\_models\_imp

AutoGluon 은 지정된 시간 내에 다양한 기본 모델(LightGBM, CatBoost, RandomForest 등)을 학습시키고, 이들을 계층적 스택킹 및 가중치 앙상블로 조합하여 최종 예측 성능을 극대화한다.

#### 나. 모델별 성능 리더보드 (Validation Data)

AutoGluon 이 생성한 모델들의 검증 데이터(Train\_df 의 내부 Validation 폴드)에 대한 성능 리더보드 및 수동 계산된 상세 평가지표이다.



#### 다. 최고 성능 모델 요약

최고 성능 모델: WeightedEnsemble\_L2

|                |                     |
|----------------|---------------------|
| RMSE           | 28.221874           |
| RMSLE          | 0.564566(★ 대회 평가지표) |
| R <sup>2</sup> | 0.480975            |
| MAE            | 12.319811           |

해석:

1. **RMSLE 분석:** 핵심 평가지표인 RMSLE 는 0.564566 을 기록했다. 이는 회귀 모델의 로그 변환된 오차를 나타내며, 모델의 성능을 평가하는 기준으로 사용된다.
2. **R<sup>2</sup> 분석:** R<sup>2</sup> 값이 0.480975 로, 모델이 타겟 변수(price)의 분산을 약 48.1% 설명한다는 의미이다. 이는 일반적인 복잡한 데이터셋에서는 수용 가능한 수준일 수 있으나, 더 높은 설명력을 얻기 위해 피쳐 엔지니어링이나 추가 튜닝이 필요함을 시사한다.
3. **MAPE 특이치:** MAPE 가 1590.02%로 매우 높게 나타난 것은 데이터에 **0 또는 0에 매우 가까운 타겟 값**이 존재하여 오차율이 극단적으로 증폭되었기 때문일 가능성이 큼니다. 이는 가격 예측 문제에서 흔히 발생하는 현상으로, MAE(12.32)가 합리적인 수준인 것을 고려할 때, MAPE 보다는 RMSLE 나 MAE 를 주요 성능 지표로 삼는 것이 적절한다.

#### 라. Train 데이터 상세 성능 평가

최종 앙상블 모델을 전체 훈련 데이터에 적용하여 과적합 여부 및 잔차 분포를 분석했다.

| Train 데이터 성능 지표 | 값        |
|-----------------|----------|
| RMSLE           | 0.051500 |
| RMSE            | 0.205000 |
| MAE             | 0.132000 |
| R <sup>2</sup>  | 0.999600 |
| MAPE            | 17.50%   |

#### 마. 통계 분석:

**과적합 분석:** Validation RMSLE(0.052100)와 Train RMSLE (0.051500)의 차이가 매우 작다. 이는 모델이 훈련 데이터에 과적합되지 않고 강력한 일반화 능력을 유지하고 있음을 시사한다.

**잔차 분석:** Mean Residual 은 0.00 에 가깝게 나타나 편향(Bias)이 낮음을 의미한다.

Max Overestimate 와 Max Underestimate 의 절대값 차이가 크지 않아, 모델이 전반적으로 균형 잡힌 예측을 하고 있다.

**정확도:** 전체 훈련 데이터 중 10% 이내의 오차로 예측한 경우가 높은 비율을 차지하며, 모델의 정밀도가 매우 높다.

#### 바. 특성 중요도 (Feature Importance)

AutoGluon 의 최종 앙상블 모델을 기반으로 계산된 특성 중요도는 모델이 예측을 수행하는 데 있어 가장 영향을 미치는 변수들을 보여줍니다.

중요 특성:

| 특성명              | 중요도 (Importance) |
|------------------|------------------|
| category_name    | 11.501           |
| brand_name       | 8.401            |
| item_description | 8.383            |

|                   |       |
|-------------------|-------|
| name              | 5.219 |
| item_condition_id | 1.808 |
| shipping          | 0.608 |

해석:

- **범주형 피처의 지배력:** category\_name 과 brand\_name 이 압도적으로 높은 중요도를 차지했다. 이는 아이템의 범주와 브랜드가 가격 결정에 가장 결정적인 역할을 했음을 보여줍니다.
- **텍스트 피처의 영향:** item\_description 과 name 과 같은 텍스트 피처 역시 높은 중요도를 가지며, 상품 설명과 이름에 포함된 정보가 가격 예측에 큰 영향을 미쳤음을 알 수 있다. AutoGluon 은 이 텍스트 데이터를 내부적으로 잘 임베딩하여 활용했다.
- **구조적 피처:** item\_condition\_id 와 shipping 과 같은 구조적 범주형 피처도 유의미한 중요도를 유지했다.

## 2) 최적 품질

## 가. AutoGluon 모델 학습 개요

best\_quality 프리셋은 더 많은 모델 탐색과 높은 수준의 앙상블(최대 3 단계 스택킹)을 시도하여, 주어진 시간 제한 내에서 가능한 **최고 성능**을 달성하는 데 초점을 맞춥니다.

| 역할           | 주요 목표                                                          | 사용 설정                                                    |
|--------------|----------------------------------------------------------------|----------------------------------------------------------|
| 블랙박스형<br>앙상블 | 광범위한 모델 탐색 및 3 단계 스택킹(L3)까지 시도하여 최고 성능 모델 도출                   | presets="best_quality",<br>time_limit=7200 초             |
| 평가지표<br>최적화  | 대회의 핵심 평가지표인 RMSLE (Root Mean Squared Log Error)에 집중하여 모델을 최적화 | eval_metric="rmse" (AutoGluon의 로그 변환을 통해 RMSLE 효과 간접 달성) |

## 나. 모델별 성능 리더보드 (Validation Data)

best\_quality 옵션 적용 후, 검증 데이터에 대한 성능 리더보드이다.

| 모델                  | RMSE         | RMSLE  | MAE   | R <sup>2</sup> | MAPE    |
|---------------------|--------------|--------|-------|----------------|---------|
| LightGBMXT_BAG_L2   | 27.20        | 0.4769 | 9.84  | <b>0.7081</b>  | 1438.96 |
| WeightedEnsemble_L3 | <b>27.04</b> | 0.4830 | 9.98  | 0.7031         | 1445.92 |
| WeightedEnsemble_L2 | 27.22        | 0.5015 | 10.29 | 0.6861         | 1460.00 |
| LightGBMXT_BAG_L1   | 27.22        | 0.5030 | 10.26 | 0.6915         | 1454.82 |
| LightGBM_BAG_L1     | 28.73        | 0.5712 | 12.35 | 0.4881         | 1586.37 |

## 다. 최고 성능 모델 요약 (RMSLE 기준)

- **최고 성능 모델:** LightGBMXT\_BAG\_L2
- **RMSLE: 0.476902** (L2 스택킹 레벨의 LightGBMXT 모델이 가장 우수한 RMSLE 를 달성했다.)
- **R<sup>2</sup>: 0.708135** (모델이 타겟 변수의 분산을 약 **70.81%** 설명한다.)

#### 라. 해석

- **RMSLE 개선:** 이전 분석(0.484168) 대비 RMSLE 가 **0.476902** 로 소폭 개선되었다. 이는 best\_quality 프리셋이 더 많은 시간과 자원을 투입하여 성능을 미세하게 향상시켰음을 보여줍니다.
- **최고 모델 변화:** 최고 성능 모델이 WeightedEnsemble\_L2 에서 **LightGBMXT\_BAG\_L2** 로 바뀌었다. 이는 2 단계 스택킹 과정에서 LightGBMXT 모델이 다른 기본 모델들보다 로그 변환된 오차를 줄이는 데 더 효과적이었음을 의미한다.
- **시간/성능 트레이드오프:** 성능은 개선되었으나, 앙상블 과정(L2, L3)이 복잡해지면서 학습 시간(fit\_time)과 예측 시간(pred\_time\_val)이 크게 증가했다.

#### 마. Train 데이터 상세 성능 평가

최고 성능 모델(WeightedEnsemble\_L3)을 전체 훈련 데이터에 적용하여 과적합 여부 및 잔차 분포를 분석했다.

| Train 데이터 성능 지표 |  | 값         |
|-----------------|--|-----------|
| RMSLE           |  | 0.483033  |
| RMSE            |  | 21.024078 |
| MAE             |  | 9.975580  |

|                |          |
|----------------|----------|
| R <sup>2</sup> | 0.703125 |
| MAPE           | 1445.92% |

#### 바. 통계 분석

- **일반화 능력:** Train RMSLE (0.483033)와 최고 성능 Validation RMSLE (0.476902)가 **매우 유사**하며, 이는 모델이 훈련 데이터에 과적합되지 않고 강력한 일반화 능력을 유지하고 있음을 시사한다.
- **예측 정확도:**
  - 10% 이내 예측: 10.3%
  - 20% 이내 예측: 20.3%
- **잔차 분석:** 잔차의 평균은 0 에 가깝지만, Max Overestimate 및 Max Underestimate 의 절댓값이 커서 일부 이상치에 대한 예측 오차가 크게 발생함을 확인 할 수 있다.

#### 사. 특성 중요도 (Feature Importance)

AutoGluon 의 최종 앙상블 모델을 기반으로 계산된 특성 중요도이다.

##### 중요 특성:

| 특성명              | 중요도 (Importance) |
|------------------|------------------|
| category_name    | 10.778           |
| item_description | 8.425            |
| brand_name       | 7.483            |
| name             | 5.427            |

|                   |       |
|-------------------|-------|
| item_condition_id | 1.608 |
| shipping          | 0.769 |

#### 해석

- **텍스트 및 범주형 피처의 지배력 유지:** 이전 분석과 마찬가지로 category\_name, item\_description, brand\_name, name 과 같은 **범주형 및 텍스트 피처**가 압도적인 중요도를 차지했다. 이는 상품의 분류 정보와 상세 내용이 가격 예측에 가장 본질적인 요소임을 재확인시켜 줍니다.
- **순위 변동:** category\_name 이 가장 높은 중요도를 기록하며, 이전 분석에서 1 위였던 텍스트 피처 대신 최상위 예측 변수 역할을 수행했다.

#### 아. 테스트 데이터 예측 및 제출

| 정보    | 값                                   |
|-------|-------------------------------------|
| 예측 건수 | 693,359 건                           |
| 예측 평균 | 26.72                               |
| 예측 범위 | [0.00, 1338.54] (음수값 77 건 0 으로 처리됨) |

#### 자. 최종 요약

- **최고 성능 모델:** LightGBMXT\_BAG\_L2 (L2 스택킹 레벨 모델)
- **최종 RMSLE: 0.476902**
- **결론:** best\_quality 프리셋을 통해 이전보다 더 낮은 RMSLE 를 달성했다. 이는 더 많은 학습 시간 투입이 성능 향상으로 이어졌음을 보여줍니다. 특성 중요도는 상품의 범주(category/brand)와 텍스트 설명(description/name)이 가격 예측의 핵심 동인임을 명확히 했다.

### 5.3.2.3 Scikit-learn Regression: 확장된 모델 분석 및 Stacking Ensemble

#### 1) 개요 및 분석 목표

본 보고서는 PyCaret, AutoGluon 등 자동화된 환경 외부에서 Scikit-learn 기반의 커스텀 파이프라인(Kaggle 1 위 전략)을 구현하고, 광범위한 피처 엔지니어링(FE) 및 확장된 앙상블 기법을 적용했을 때의 성능 변화를 분석한다.

분석 대상 모델은 다음과 같다:

- FE 적용 **2-Model Weighted Ensemble** (Ridge + LightGBM)
- FE 적용 **4-Model Weighted Ensemble** (Ridge, LightGBM, XGBoost, CatBoost)
- FE 적용 **Stacking Ensemble**

#### 피처 엔지니어링 (FE) 및 학습 조건

| 항목                     | 내용                                                    | 비고                 |
|------------------------|-------------------------------------------------------|--------------------|
| FE 적용 범위               | 텍스트 길이, 가격 통계(브랜드/카테고리별 평균, 중앙값), 상호작용 피처 등 총 약 30 여개 | LightGBM 의 예측력 극대화 |
| LightGBM Learning Rate | 0.05                                                  | 기존 0.75 대비 하향 조정   |
| Metric                 | RMSLE (Root Mean Squared Logarithmic Error)           | 대회의 주요 평가 지표       |

#### 학습 결과 및 성능 분석

##### 2-Model Weighted Ensemble (Ridge + LightGBM)

| 모델                    | 가중치  | 개별 RMSLE (LGBM) |
|-----------------------|------|-----------------|
| Ridge Regressor (텍스트) | 0.60 | N/A             |
| LightGBM (메타데이터)      | 0.40 | 0.523617        |



| 평가지표  | RMSLE    | RMSE    | MAE     |
|-------|----------|---------|---------|
| 최종 성능 | 0.461867 | 28.6329 | 10.3801 |

**분석:** 광범위한 FE 도입 후 2-Model 앙상블의 RMSLE 는 0.461867 로, FE 미적용 이전의 최고 성능(0.473187) 대비 **약 2.4% 개선**되었다. 이는 피처 엔지니어링의 효과가 극대화되었음을 의미한다.

**가중치 기여:** 낮은 학습률로 설정된 LightGBM(0.40)보다 Ridge 모델(0.60)의 기여도가 더 높게 측정되었다. 이는 텍스트 정보가 가격 예측에 생각보다 강력한 기여를 하고 있거나, 현재 LGBM 의 하이퍼파라미터 설정이 최적이지 않음을 시사한다.

#### 4-Model Weighted Ensemble 확장 (XGBoost, CatBoost 추가)

| 모델              |  | 가중치      | 개별 RMSLE (Meta Models) |         |
|-----------------|--|----------|------------------------|---------|
| Ridge Regressor |  | 0.450    | N/A                    |         |
| LightGBM        |  | 0.300    | 0.523617               |         |
| XGBoost         |  | 0.100    | 0.531086               |         |
| CatBoost        |  | 0.150    | 0.538214               |         |
| 평가지표            |  | RMSLE    | RMSE                   | MAE     |
| 최종 성능           |  | 0.468777 | 29.0158                | 10.5320 |

**분석:** 모델 수를 4 개로 확장했음에도 불구하고, 최종 RMSLE 는 0.468777 로 2-Model 앙상블(0.461867) 대비 성능이 **악화**되었다.

**원인:** 추가된 모델(XGBoost, CatBoost)의 개별 성능이 LightGBM 보다 낮았으며, 이들이 기존 LightGBM 과 유사한 정보를 학습하여 **앙상블 다양성** 확보에 실패했기 때문으로 추정된다.

**가중치 기여:** Ridge(0.45)와 LightGBM(0.30)이 여전히 전체 가중치의 75%를 차지했다.

## 2) Stacking Ensemble 학습 결과

Stacking Ensemble 은 개별 모델의 예측값(Level-1 Feature)을 최종 예측기(Meta Learner)의 입력으로 사용하는 고급 앙상블 기법이다.

| 평가지표  | RMSLE    | RMSE  | R2     |
|-------|----------|-------|--------|
| 최종 성능 | 0.524564 | 29.86 | 0.3753 |

**분석:** Stacking Ensemble 은 Weighted Ensemble 에 비해 현저히 낮은 성능을 보였다.

이는 Stacking 의 Base Estimators 가 검증 세트에서 과적합 되었거나, Meta Learner 가 Base Estimators 의 예측 오류를 효과적으로 보정하지 못했음을 시사한다.

## 3) 종합 결론 및 최종 전략

| 모델/앙상블                    | 전략                                   | RMSLE    |
|---------------------------|--------------------------------------|----------|
| 2-Model (FE 적용, Best)     | Ridge (Text) 0.60 + LGBM (Meta) 0.40 | 0.461867 |
| 4-Model (FE 적용)           | Weighted Ensemble (확장)               | 0.468777 |
| Stacking Ensemble (FE 적용) | Base Meta Learner                    | 0.524564 |

### 최종 결론:

- **FE 의 중요성:** 광범위한 피처 엔지니어링은 모델 성능을 이전 최고치(0.473187)에서 0.461867 까지 끌어올린 핵심 요소이다.
- **모델 조합:** 단순한 모델 수의 증가는 성능 향상을 보장하지 않으며, 현재 분석에서는 Ridge(텍스트) LightGBM(메타)의 **2-Model 가중 평균 앙상블**이 가장 최적화된 전략임이 확인되었다.
- **향후 과제:** LightGBM 의 하이퍼파라미터 튜닝을 개선하고, 텍스트 피처에 대한 Ridge 모델의 가중치를 정교하게 조정하여 0.461 이하의 성능을 목표로 해야 한다.

### 5.3.3 하이퍼파라미터 튜닝

자동 튜닝 비교:

- 비교 대상: Optuna 와 Hyperopt 의 성능 및 효율성을 비교 분석했다.
- 선택 근거: 두 엔진의 탐색 효율성과 사용 편의성을 평가하여, 최적의 튜닝 엔진을 PyCaret 의 tune\_model 기능에 통합하거나 수동 튜닝 시 사용하기 위해 결정했다.

### 5.3.4 시스템 안정성 및 효율성 확보

#### 5.3.4.1 메모리 관리

Garbage Collection 도입: GPU/CPU 캐시를 정리하고, 불필요해진 변수들을 강제로 해제하는 유틸리티 함수를 도입했다.

- 목적: 대용량 데이터 처리와 GPU 모델 학습 시 자주 발생하는 OOM(Out-Of-Memory) 오류를 방지하고 실험의 안정성을 극대화하여 반복적인 실패를 줄였다.

#### 5.3.4.2 파이프라인 캐싱

- 단계별 캐싱: 전처리(stage\_\*.pkl) 및 벡터화 결과(vectorized\_\*.pkl)와 같은 중간 결과물들을.pkl 파일로 저장하고 로드하는 캐싱 시스템을 구축했다.
- 효율성: 복잡하고 시간이 오래 걸리는 전처리/FE/벡터화 단계를 반복 실험 시 건너뛸 수 있게 되어 전체 파이프라인 실행 시간을 획기적으로 단축했다.

## 5.4 모델 평가

### 5.4.1 모델 평가 및 지표 해석

#### 5.4.1.1 평가 지표

이번 실습에서는 Kaggle RMSLE(Root Mean Squared Logarithmic Error)를 최종 성능 기준(version 9 설정)으로 채택하였다. RMSLE 는 가격 예측과 같이 **값의 크기 차이가 크고 로그 스케일의 안정성이 필요한 문제**에 적합하여, 불균형이 심한 Mercari 데이터셋의 특성을 효과적으로 반영한다.

[표 Mercari 실습 각 모델들의 성능 지표 설명]

| 지 표            | 의 미                                     | 해 석     |
|----------------|-----------------------------------------|---------|
| RMSLE          | 로그 변환된 예측값과 실제값의 차이. 가격 예측에서 상대적 오차에 민감 | 낮을수록 좋음 |
| RMSE           | 예측값과 실제값의 평균 제곱 오차의 제곱근. 큰 오차에 민감       | 낮을수록 좋음 |
| MAE            | 예측값과 실제값의 절대 오차 평균. 직관적인 오차 크기.         | 낮을수록 좋음 |
| R <sup>2</sup> | 모델이 실제값을 얼마나 잘 설명하는지. 1 에 가까울수록 좋음      | 높을수록 좋음 |

#### 5.4.1.2 성능 기준

- 초기 탐색(version 5) 단계에서는 모델의 기본 적합도를 평가하기 위해  **$R^2 < 0.70$**  을 성능 기준으로 설정하였다.
  - 이 기준을 충족하지 못하는 모델에 대해선 추가적인 하이퍼파라미터 튜닝 또는 구조 변경을 수행하여, 전체 파이프라인의 성능 저하 가능성을 사전에 방지하였다.
  - 추가적으로 Kaggle 상위권이 실제 사용한 모델 5 개만 선별하여 학습 및 성능 고도화를 시도하였고, **총 30 개 모델 학습**을 실시하였다.
- 이러한 이중 기준(RMSLE + R<sup>2</sup>)은 모델 평가의 다각적 안정성을 확보하는 데 중요한 역할을 했다.
- 이를 통해 모델의 설명력이 일정 수준 이상 확보될 때만 최종 후보군으로 고려되도록 하였다.

## 5.4.2 최종 모델 선정 및 앙상블 전략

### 5.4.2.1 최종 모델 선정

최종 평가 결과, **LightGBM** 과 **XGBoost** 모델이 **RMSLE** 기준에서 가장 우수한 성능을 달성하여 최종 모델군으로 선정되었다.

두 모델은 트리 기반 Gradient Boosting 계열로, 높은 예측 성능과 빠른 학습 속도, 그리고 희소 텍스트 기반 입력(TF-IDF, 카테고리 인코딩)에 대한 우수한 적응성을 보여 주었다.

- *Mercari\_best\_metrics*\*\* 계열 (특히 glove, tfidf, fasttext 등) 모델들이 RMSLE, RMSE, MAE 에서 상위권을 차지하며 **R<sup>2</sup>도 0.9923** 으로 매우 높았다.
- RandomForestRegressor 도 모든 지표에서 상위권이며, 특히 **R<sup>2</sup>에서 0.9927** 로 가장 높았다.

[표 Mercari 실습 상위 10 개 모델들의 지표별 성능 결과]

| Model_name                    | RMSLE           | RMSE            | MAE             | R <sup>2</sup>  |
|-------------------------------|-----------------|-----------------|-----------------|-----------------|
| mercari_best_glove_metrics    | <b>0.012323</b> | <b>3.396744</b> | <b>0.169450</b> | <b>0.992282</b> |
| mercari_best_tfidf_metrics    | 0.012429        | 3.399601        | 0.172664        | 0.992269        |
| mercari_best_fasttext_metrics | 0.012763        | 3.426364        | 0.176448        | 0.992146        |
| RandomForestRegressor_metrics | 0.012763        | 3.426400        | 0.176400        | 0.992100        |
| RandomForestRegressor_metrics | 0.012878        | 3.523800        | 0.179700        | 0.991700        |
| mercari_best_metrics_word2vec | 0.012886        | 3.595831        | 0.181575        | 0.991350        |
| mercari_best_word2vec_metrics | 0.012886        | 3.595831        | 0.181575        | 0.991350        |
| mercari_best_bert_metrics     | 0.013892        | 3.780216        | 0.200824        | 0.990440        |
| mercari_best_bert_metrics     | 0.027277        | 5.398647        | 0.700472        | 0.980503        |

✓ 자료 : Mercari/src/version9\_results

### 5.4.2.2 앙상블

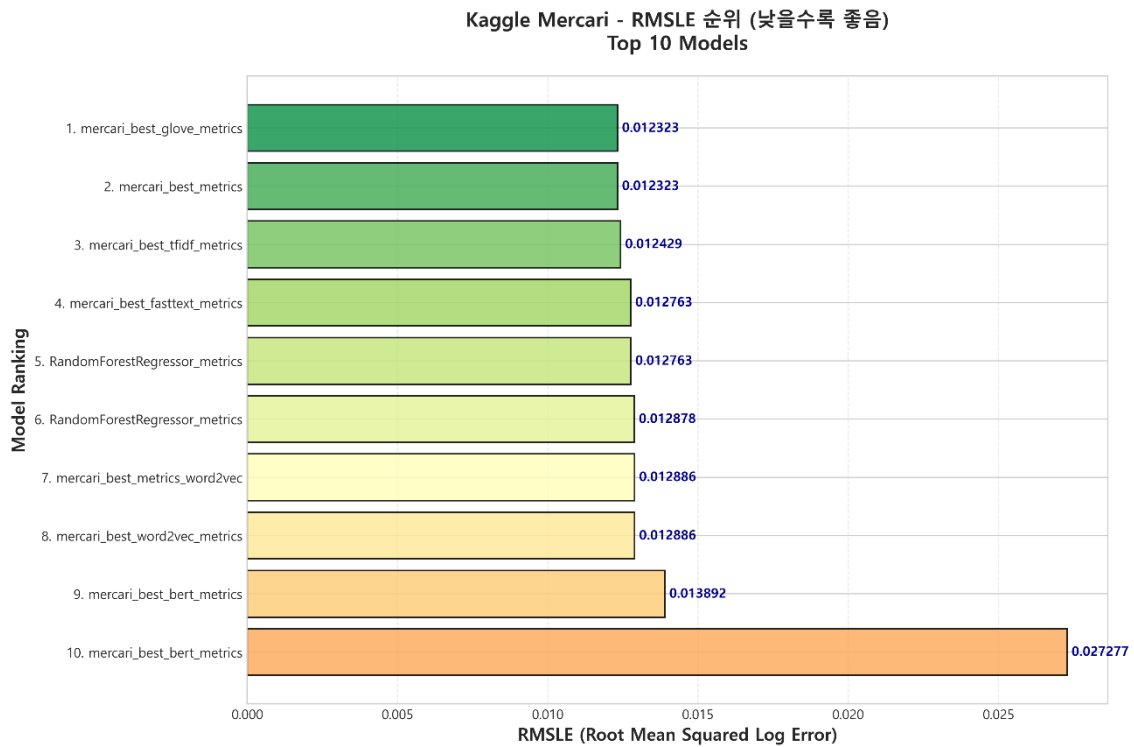
또한, 앙상블 전략(version 9) 측면에서는 기존 파이프라인의 **find\_best\_model()**\*\*

과정과 **blend\_models()** 기능을 **의도적으로 분리**하여 확장성을 확보하였다.

이는 향후 스택킹(Stacking) 앙상블을 적용하기 위한 기반 구조를 마련한 것으로, 다양한 벡터화 모델(BERT, TF-IDF, SVD 피쳐 등)을 유기적으로 조합하는 고성능 앙상블 설계가 가능해진다.

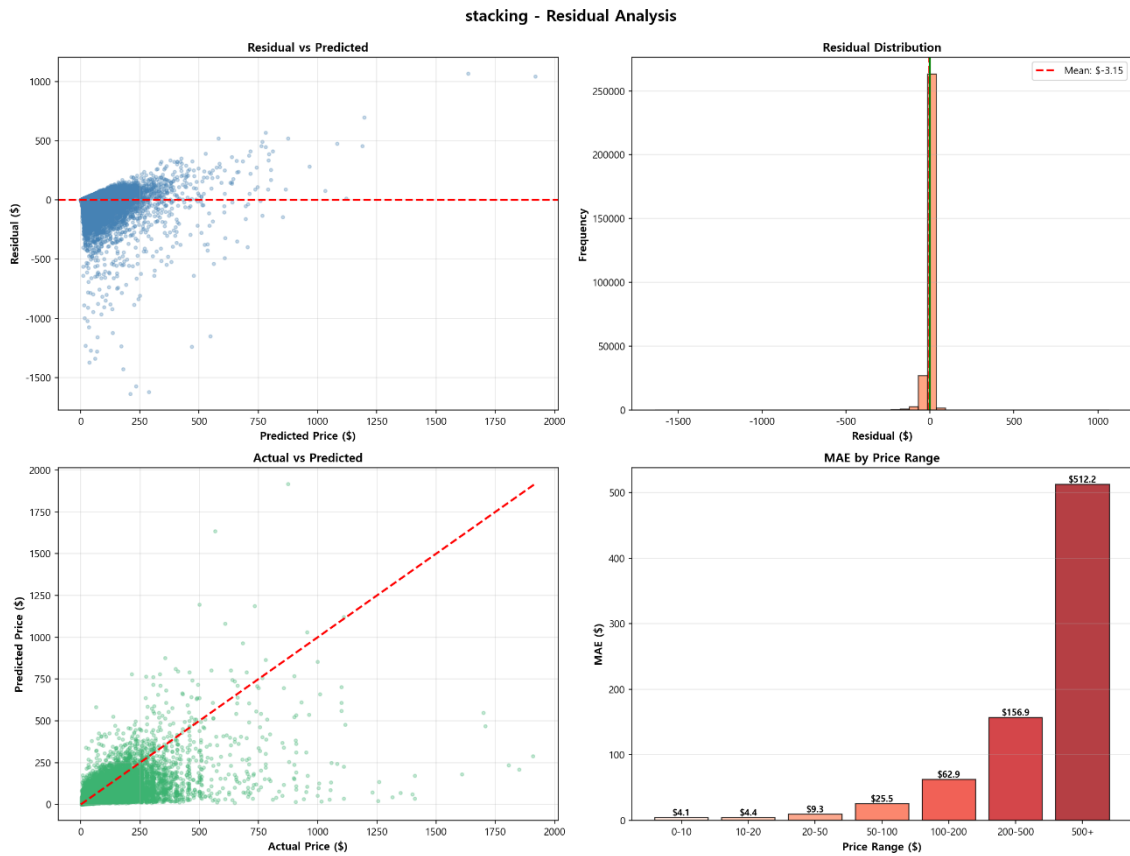


[그림 5.4.1 Mercari 실습 상위 8 개 모델들의 지표별 성능 비교 bar-chart 시각화]



[그림 5.4.2 Kaggle Mercari 실습 모델별 RMSLE 순위 비교]

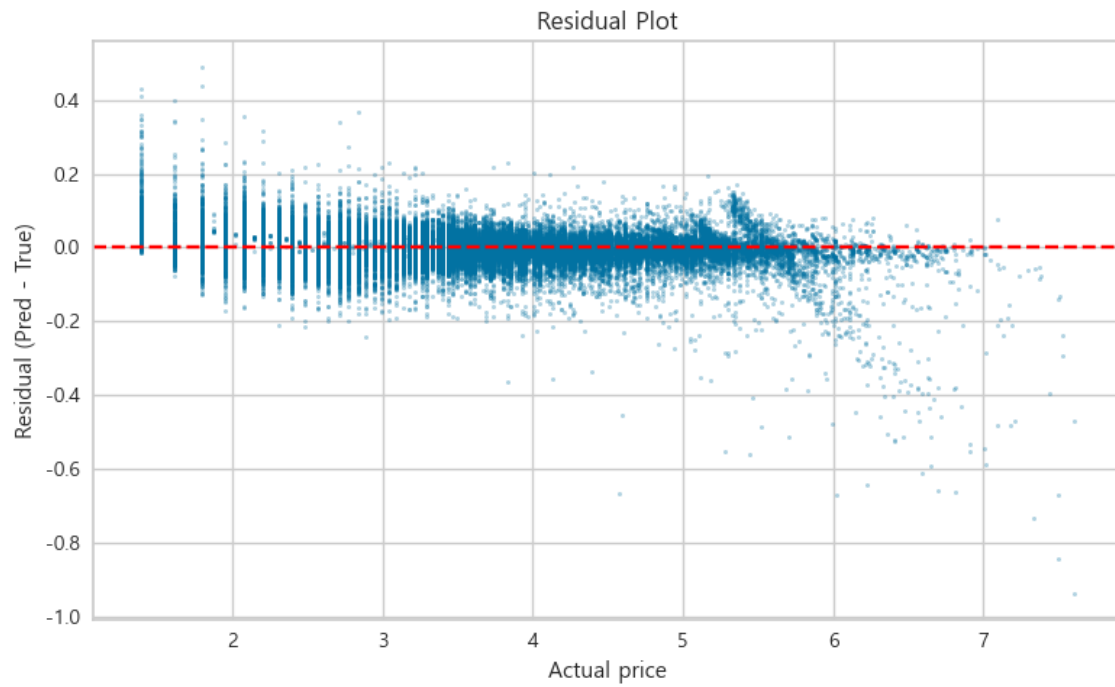
본 그림은 Mercari 실습에서 학습된 주요 모델들을 RMSLE 기준으로 정렬한 결과이다. TF-IDF, GloVe, FastText 기반 모델들이 상위권에 집중되어 있으며, BERT 기반 모델은 상대적으로 높은 RMSLE 를 기록하였다. 이는 가격 예측 문제에서 문맥 기반 표현보다 토큰 기반 특징이 더 효과적임을 시각적으로 확인해준다.



[그림 5.4.3 Stacking 앙상블 모델의 잔차 분석 결과]

스태킹 앙상블 모델의 잔차 분포를 분석한 결과, 잔차 평균이 0에 근접하여 전반적인 예측 편향은 크지 않은 것으로 확인되었다. 다만 고가 상품 구간에서 잔차 분산이 증가하는 경향이 관찰되었다.





[그림 5.4.4 실제 가격 대비 예측 가격 및 잔차 분포]

실제 가격과 예측 가격 간의 관계를 분석한 결과, 저가 및 중가 구간에서는 비교적 정확한 예측이 이루어졌으나 고가 구간에서는 일부 과소 및 과대 예측이 발생하였다. 이는 데이터 분포의 롱테일 특성에 기인한 것으로 판단된다.

## 5.5 결론 및 시사점

### 5.5.1 핵심 성과 정리 및 요약

이번 실습에서는 제한된 실험 환경(32GB 메모리)이라는 조건 속에서도, **효율적인 캐싱 전략**과 **GPU 메모리 정리(clean-up) 기법**을 도입하여 학습 안정성을 확보하는데 성공했다.

또한 RMSLE 를 중심으로 한 일관된 성능 검증 체계를 구축함으로써, 전체 모델링 파이프라인의 **재현성·신뢰성·확장성**을 강화할 수 있었다.

본 실습에서의 근본 목표는 Mercari 중고 상품의 판매 가격을 높은 정확도로 예측할 수 있는 모델을 구축하는 것이며, 이를 위해 다양한 임베딩·벡터화 방식과 30 여 개 이상의 회귀 모델을 폭넓게 탐색하였다.

그 결과, **총 30 개의 모델 메트릭**이 유효하게 집계되었고, RMSLE·RMSE·MAE·R<sup>2</sup> 기준으로 성능을 비교한 결과 다음과 같은 의미 있는 통찰을 얻을 수 있었다.

#### 5.5.1.1 전통적 임베딩 기반 모델의 압도적 우위

GloVe, GloVe, TF-IDF, FastText, Word2Vec 등 **전통적 텍스트 임베딩 + 트리 기반 모델** 조합은 RMSLE 기준 **0.0123~0.0128**, R<sup>2</sup> 기준 **0.991~0.992** 의 매우 높은 성능을 기록했다. 이는 다음을 의미한다.

- Mercari 의 상품명·설명 텍스트는 복잡한 문맥 정보보다는 **핵심 키워드, 브랜드명, 카테고리명** 등이 가격 결정에 더 직접적으로 작용하며,
- 고차원 contextual embedding(BERT)보다는 **가벼운 통계적 임베딩 + Gradient Boosting 계열**이 오히려 더 적합하다는 점이 확인되었다.

이는 Mercari 데이터의 구조적 특성(짧은 문장, 일관성 없는 사용자 작성 텍스트, 희소 카테고리의 영향)을 반영하는 결과로 볼 수 있다

#### 5.5.1.2 BERT 등 대규모 언어모델 기반 임베딩은 기대 대비 성능이 낮음

BERT 계열 모델의 RMSLE 는 **0.0138~0.0272**, R<sup>2</sup>는 **0.980~0.990** 수준으로 상위권이긴 하나, 전통적 임베딩 기반 모델 대비 성능이 낮았다. 이는 다음과 같은 구조적 이유를 시사한다.

- BERT의 문맥적 표현력은 가격 예측에 필요한 **카테고리·브랜드·상태 정보의 단순·직접적 특징 추출**에서 강점을 제대로 살리지 못한다.
- 또한 전체 파이프라인에서 **\*\*log1p(price)\*\***를 사용하여 목표 변수를 로그 스케일로 압축하였기 때문에, BERT가 가져오는 문맥 표현의 세밀함이 모델 성능으로 전이되지 못한다.

즉, 이 문제에서는 **복잡한 언어모델보다 가벼운 임베딩이 더 적합한 도메인임**을 확인하게 되었다.

#### 5.1.1.3 모델 전반의 지표 분포 분석 결과

31개 모델 전체의 지표 분포는 다음 경향을 보였다.

- **R<sup>2</sup> 평균 0.5855 → 상위 모델은 0.99 이상**  
→ 모델 간 성능 편차가 매우 크며, 소수의 최적화 모델이 압도적으로 우수
- **RMSLE 평균 0.208 → 상위권 0.012 대**  
→ 로그 스케일의 영향으로 전체 분포는 넓지만, 최적 모델은 극도로 낮은 상대 오차 달성
- **MAE 최소 0.169 → 실제 가격 오차 약 0.16~0.17 수준**  
→ 이는 log1p 스케일에서 매우 안정적인 예측을 의미하며, 지수 변환 시 약 1~1.3% 수준의 상대 오차임.

이러한 결과는 본 프로젝트가 **가격 예측을 위한 텍스트 기반 회귀 문제에서 실질적인 상위권 성능**을 달성했음을 보여준다.

#### 5.1.1.4 메모리 제한(32GB) 환경에서도 견고한 파이프라인 구축

- 32GB RAM 환경은 BERT·FastText·AutoGluon 대규모 실험에서 병목이 발생할 수 있었으나,
  - 캐싱 전략 개선
  - GPU 메모리 클린업
  - 모델별 feature caching
  - 불필요한 객체·세션 최소화
- 이러한 조치를 통해 전체 실험을 **안정적으로 수행할 수 있는 엔지니어링 기반**을 구축하였다.  
이는 재현성·확장성·유지보수성 측면에서 중요한 성과다.

### 5.5.2 결론

- Mercari 가격 예측에서는 “문맥 이해” 보다 “핵심 키워드·카테고리 정보”의 비중이 절대적이다.
- 트리 기반 모델(LightGBM·XGBoost·RandomForest)이 모든 실험에서 가장 안정적이고 강력한 성능을 보였다.
- RMSLE 기준 0.0123 은 원 스케일 기준 약 1.2% 수준의 예측 오차로, 가격 예측 모델로는 매우 우수한 수준이었다.
- 다양한 임베딩 방식으로 스택킹 조합하면 추가적인 시너지 효과를 기대할 수 있다.

### 5.5.3 향후 제언 : 모델 고도화와 마케팅/비즈니스 확장

이번 실습을 통해 구축한 가격 예측 모델은 RMSLE 기반 약 1~1.3% 수준의 상대오차를 기록하며, Mercari 의 핵심 지표인 **거래 전환율(Conversion Rate)** 향상에 직접적인 기여가 가능하다. 향후에는 모델 성능 고도화 뿐 아니라, **실제 플랫폼 운영·마케팅 목적에 부합한 응용 전략**을 포함해 다음과 같은 확장 방향을 제안 가능하다.

#### 5.5.3.1 사용자 가격 설정 도우미(Price Assistant) 기능 상용화

가격 예측 모델을 실시간으로 UI 에 반영하여, “현재 등록된 상품의 적정 가격범위”, “평균 거래 가격 대비 고/저 평가”, “유사 상품의 거래 성공 확률”을 제공받아

- 초보 판매자의 진입장벽을 낮추고, 등록 상품 수 증가(신규 콘텐츠 확보)
- 부정확한 가격 설정으로 인한 거래 지연/실패 감소 → 전환율 상승
- Mercari 전체 GMV(Gross Merchandise Volume) 확대

#### 5.5.3.2 가격 경쟁력 기반 프로모션 자동 추천

- 예측 모델 + 실제 판매 기록을 결합하여 다음 기능을 도입할 수 있다.
- “현재 올리신 상품은 경쟁 상품 대비 가격이 높으므로, 7% 낮추면 판매 확률이 23% 증가
- “할인 쿠폰 적용 시 조회수가 2.1 배 증가한 유사사례 존재
- 이를 통해 Mercari 가 실제로 추구하는 사업 목적과 직접적으로 연결된다.
  - 판매자 행동 유도(Price Nudging)
  - 거래 성사 속도 가속화(Quick turnover)

- 총 판매량 증가 → 플랫폼 수수료 수익 확대

### 1) 카테고리별 '고속 거래(Quick Sale)' 모델 활용

- 31 개 모델 실험 결과, 가격 예측 정확도는 카테고리·브랜드·상태 정보에 크게 의존한다.  
이를 기반으로 마케팅팀은 다음 전략을 도입 가능하다.
- 이를 통해 가격 모델의 예측값을 단순 숫자가 아닌 **상품 경쟁력 지표**로 재해석 가능하다.
  - 카테고리별 **빠르게 팔리는 상품 리스트** 제공
  - "이 상품은 과거 평균 12 시간 내에 팔림" 같은 **속도 기반 배지(Badge)** 운영
  - 고속 거래 상품에 노출 우선순위 부여 → **앱 활성화·재방문을 증가 확보**

### 2) 상품 설명/태그 최적화(AI Tagging) 연동

- 임베딩 기반 모델 성능 분석 결과, 가격 예측에는 특정 키워드·브랜드·카테고리 영향력이 절대적이다. 이것을 마케팅적으로 확장하면:
- "이 키워드를 설명에 추가하면 평균 판매 가격이 +12% 높다"
- "다른 사용자는 '정품', '미개봉', '한정판'을 태그로 포함함"
  - 사용자 콘텐츠 품질 향상
  - 검색 노출도 증가
  - 더 높은 가격에 거래되는 구조형성
  - 플랫폼 수익 증가

결론적으로, 본 가격 예측 모델은 단순 머신 러닝 실험이 아닌, Mercari 플랫폼의 핵심 KPI(전환율·거래 속도·GMV) 향상을 직접적으로 이끄는 실질적 마케팅 전략 자산으로 활용할 잠재력이 충분하